

CS 182 — Deep Neural Networks

Fall 2025

Contents

Lecture 0: Introduction/Administrivia	6
Lecture 1: Basic Principles	14
1.1 Recap: Function Approximation with Neural Networks	14
1.1.1 The Problem: Learning a Function from Samples	14
1.1.2 Piecewise Linear Approximation	14
1.1.3 Neural Network Representation	15
1.1.4 The ReLU Activation Function	15
1.1.5 Mathematical and Block Diagram Formulation	15
1.2 Machine Learning vs. Deep Learning Paradigms	15
1.2.1 Classical Machine Learning	15
1.2.2 Supervised Learning	16
1.2.3 Unsupervised Learning	16
1.2.4 Deep Learning Paradigms	16
1.2.5 Supervised and Unsupervised Deep Learning	16
1.2.6 Foundation Models	17
1.3 Optimization for Deep Learning	17
1.3.1 Empirical Risk Minimization (ERM)	17
1.3.2 The True Goal and Key Assumption	17
1.3.3 Challenges in Optimization	17
1.3.4 Regularization and Hyperparameters	18
1.4 Gradient Descent	18
1.4.1 The Algorithm	18
1.4.2 Analysis for the Least Squares Problem	19
1.4.3 Convergence and Optimal Learning Rate	19
1.5 Regularization and Gradient Descent	20
1.5.1 Ridge Regression and SVD	20

Lecture 2: Optimization: implicit regularization, SGD, and momentum	21
2.1 Implicit Regularization of Gradient Descent	21
2.1.1 Recap: Convergence of Gradient Descent for Linear Regression	21
2.1.2 Ridge Regression in SVD Coordinates	21
2.1.3 Implicit Regularization via Gradient Descent and Early Stopping	22
2.2 Stochastic Gradient Descent (SGD)	23
2.2.1 The SGD Algorithm	23
2.2.2 Properties of SGD	24
2.3 Momentum	24
2.3.1 Motivation: The Problem of Oscillation	24
2.3.2 An Exponential Moving Average of Gradients	24
2.4 A Look at SGD Convergence (Underdetermined Case)	25
2.4.1 Problem Setup	25
2.4.2 Change of Variables and Simplification	26
2.4.3 SGD Update and Proof Strategy	26
 Lecture 3: Optimization: Adam, locally linear perspective, features	 27
3.1 Convergence of SGD with Constant Step Size	27
3.1.1 Problem Setup	27
3.1.2 Change of Coordinates	27
3.1.3 Singular Value Decomposition (SVD) Basis	27
3.1.4 SGD Update and Convergence Proof	28
3.2 The Adam Optimizer	29
3.2.1 Motivation: The Challenge of Gradient Descent	29
3.2.2 Normalizing Gradients: Sign SGD	30
3.2.3 Adam: Adaptive Moment Estimation	30
3.2.4 Properties of Adam	31
3.2.5 AdamW: Adam with Decoupled Weight Decay	31
3.3 Neural Networks as Linearized Models	31
3.3.1 Perspective 1: Feature Extraction	32
3.3.2 Perspective 2: Taylor Expansion	32

Lecture 4: Optimizers	33
4.1 Linear Perspectives on Neural Networks	33
4.1.1 A Second Perspective: Linearization via Taylor Expansion	33
4.2 Revisiting Descent Algorithms	33
4.2.1 The Core Tension in Optimization	34
4.2.2 Optimization as a Constrained Problem	34
4.3 Standardization and Normalization	35
4.3.1 Why Standardize Data?	35
4.3.2 The Danger of Unstandardized Data: A ReLU Example	36
4.4 Weight Initialization Strategies	36
4.4.1 Why Not Initialize All Weights to Zero?	36
4.4.2 Xavier (Glorot) Initialization	37
4.4.3 He Initialization	38
 Lecture 5: Optimizers: insights from induced matrix norms	 39
5.1 Motivation: Why Better Optimizers?	39
5.1.1 The Practical Importance of Optimization	39
5.1.2 The State of the Art and Its Challenges	39
5.1.3 Lazy Training: A Case for Better Optimizers	39
5.2 A Local Linear Perspective on Optimization	40
5.3 A Recipe for Optimizers from Norms	41
5.3.1 Choice 1: The ℓ_∞ Norm (Sign SGD)	41
5.3.2 Choice 2: The ℓ_2 Norm (Gradient Descent)	41
5.4 Incorporating Network Structure: Matrix-Based Optimization	42
5.4.1 The Shampoo Optimizer and the Spectral Norm	42
5.4.2 Deriving the Shampoo Update Step	43
5.5 Towards a Principled Choice of Norm for Deep Learning	43
5.5.1 Recapping Xavier Initialization and Norms	43
5.5.2 The Root-Mean-Square (RMS) Norm	44
5.5.3 The Induced RMS-to-RMS Norm	44
5.6 Conclusion and a Look Ahead	44

Lecture 6: muP: maximal update parameterization	46
6.1 Recap: The RMS to RMS Norm	46
6.2 The Optimizer Recipe and Layer-Specific Rates	46
6.2.1 The General Recipe	46
6.2.2 Using the RMS to RMS Norm	47
6.3 Deriving an Optimal Step-Size Scaling for Adam/SignSGD	47
6.3.1 Analysis with Mini-Batch Size 1	48
6.4 Maximal Update Parameterization (μ P)	49
6.4.1 The Hyperparameter Transfer Problem	49
6.4.2 Analogy: The Central Limit Theorem	49
6.5 A Principled View: Conditions for Stable Feature Learning	49
6.5.1 Achieving the Desiderata	50
Lecture 7: Optimizers: MuON	51
7.1 Recap: Maximal Update Parametrization (MUP)	51
7.1.1 A Spectral Perspective on Feature Learning	51
7.1.2 Deriving Scaling Conditions	52
7.2 Muon: Momentum Orthogonalized by Newton-Schulz	53
7.2.1 Motivation: State-of-the-Art Performance	53
7.2.2 Revisiting the Optimizer Recipe and Shampoo	53
7.2.3 Key Idea 1: RMS-to-RMS Norm Optimization	54
7.2.4 Key Idea 2: Efficient Semi-Orthogonalization via Newton-Schulz	54
7.2.5 The Muon Algorithm and Implementation Details	55
7.3 Conclusion and Future Directions	56
Lecture 8: Conv-nets: basics	57
8.1 Introduction to Architectures	57
8.1.1 MLPs vs. CNNs: A High-Level View	57
8.2 Motivation and Origins of CNNs	57
8.2.1 Input Data: Images	57
8.2.2 Typical Computer Vision Tasks	58
8.3 Core Principles of CNNs	58

8.3.1	Expressivity: Building in Prior Knowledge	58
8.3.2	Practicalities: Getting the Network to Work	59
8.4	The Convolution Operation in Deep Learning	60
8.4.1	Convolution vs. Cross-Correlation	60
8.4.2	Mechanics of a Convolutional Layer	60
8.5	Architectural Details and Hyperparameters	61
8.5.1	Handling Boundaries: Padding	61
8.5.2	Receptive Field	61
8.5.3	Downsampling via Stride	62
Lecture 9: Pooling/Downsampling, Data Augmentation, Normalization Layers		63
9.1	Recap and Details of Convolutional Layers	63
9.1.1	The Convolution Operation	63
9.1.2	Understanding The Number of Parameters	63
9.2	Backpropagation in CNNs	65
9.2.1	The Conceptual Approach	65
9.2.2	Practical Considerations: Sum vs. Average	65
9.3	Pooling and Downsampling	65
9.4	Classic CNN Architectures	66
9.5	Data Augmentations	66
9.5.1	Basic Augmentations	67
9.5.2	Aggressive Augmentations for Robustness	67
9.6	Training Stability: Normalization Layers	67
9.6.1	RMS Norm Layer	68
9.6.2	Batch Normalization (BatchNorm)	68

Lecture 0: Introduction/Administrivia

Today's Agenda

1. Introductions
2. Administrative Details
3. Introduction to Deep Learning

Course Philosophy

A core tenet of this class, inspired by the late Professor Patrick Winston of MIT, is that *"All great ideas are simple. Good ideas are complicated."* Our goal is to distill complex topics into their simplest, most elegant forms. To appreciate this simplicity, a strong foundation in powerful mathematical tools is essential. Just as the Singular Value Decomposition (SVD) simplifies complex matrices by projecting them into the right coordinates, we will use the language of mathematics to find clarity and beauty in deep learning.

Course Administration

Instructors

- **Prof. Giri Krishnan:** PhD from UC Berkeley, previously a researcher at Microsoft Research.
- **Prof. Anant Sahai:** Professor at UC Berkeley, has taught this course multiple times.
- **Office Hours:** Right after class in 400 Cory Hall.

Course Staff

- **GSI:** Sultan Daniels (3rd year PhD student)
- **uGSIs:** Patrick (4th year EECS & Applied Math), Alex (3rd year EECS & Math)

Resources

- **Website:** eecs182.org. All official policies are documented here.
- **Ed Discussion:** The primary mode of communication. All posts are required reading. It is the fastest way to get a response from faculty and staff. For private matters, use private posts on Ed.
- **Textbook (Recommended):** *Understanding Deep Learning* by Simon Prince. This is a good, recent textbook, but the course will cover material not in the book, including recent research papers. It is a reference, not a requirement.

Enrollment

The class may expand, but this is not under the instructors' control. To prioritize engaged and committed students, we have the following policy:

- A **Homework 0** will be due next week.
- Students enrolled in the class who **do not submit Homework 0 will be dropped**.
- Students on the waitlist **must complete and submit Homework 0** to be considered for a spot in the class.

Sections and Getting Help

- **Discussion Sections:** Wednesdays at 3-4 PM and 4-5 PM. Sections are **required** and will cover important material. They start Wednesday, September 3rd. You may attend either section.
- **Getting Help:**
 1. Instructor Office Hours (after class)
 2. GSI Office Hours (TBD)
 3. Discussion Sections
 4. Ed Discussion Forum
 5. **Each other:** Form study groups! This is a fast-paced course, and working together is critical for success. Set up regular meeting times to work on homework and hold each other accountable.

Prerequisites

This course assumes **mastery** of the following subjects. This background material will not be re-taught or supported in office hours.

- **Machine Learning:** At the level of CS 189 or Stat 154.
- **Linear Algebra & Optimization:** At the level of EECS 127/227A. Optimization is the workhorse of modern deep learning.
- **Probability:** At the level of EECS 126, Data 140, or Stat 134. Probability provides the language for generalization and the intuition behind many algorithms.

This is not a "cookbook" deep learning class. We aim for a deep, fundamental understanding of the material.

Grading and Policies

There will be **no curve** in this class. Grading bins are published on the course website.

Grade Breakdown

- **Final Exam: 60%**
- **Project: 25%**
- **Participation: 15%**
- **Homework: 0%** (Nominally)

Gating Effect: You must pass the final exam to pass the class, and you must pass the project to pass the class. This is to ensure individual understanding and prevent free-riding on group projects.

Homework Policies

- **Submission:** Join GradeScope. Homeworks involve both written proofs/derivations and coding components. Submit a single PDF containing scans of your written work and a PDF of your code notebook.
- **Deadline:** Homeworks are due weekly on Friday nights. However, the GradeScope deadline is set to the end of the semester to provide automatic extensions if needed. You are responsible for keeping up.
- **Purpose:** Homework is the best way to learn the material. While worth 0%, we may look at a random subset of your homeworks if your final grade is near a boundary. Using an LLM to do your homework is like driving around the block instead of running; you miss the entire point, which is the workout (learning).

Project

- **Groups:** Groups of four are required.
- **Topics:** We will provide a few open-ended project options that build on published papers and allow you to explore near the research frontier. Graduate students from outside EECS may be permitted to propose projects aligned with their own research.
- **Compute:** We are working to secure some cloud compute resources, but they will be limited. You may need to use your own compute for ambitious projects.
- **Peer Review:** There will be a mandatory individual peer review component. Your project grade will be a combination of your group's technical submission and your individual performance as a peer reviewer. Failure to complete peer reviews will result in a zero for the project.

Participation and Citizenship (15%)

Everyone starts with a failing grade (0%) in participation. To earn credit, you must contribute to the class community.

- **Ordinary Participation:** Achieved by:
 1. Attending and actively participating in weekly discussion sections.
 2. Being actively helpful on the Ed Discussion forum.
- **Special Contributions:** You are expected to complete **at least four** of these throughout the semester. These are tasks that engage your learning and improve the class for everyone. Examples include:
 1. **LLM for Homework (max 1):** Use an LLM (e.g., Gemini) to solve a homework problem. Document the entire interaction trace and provide an annotated report critiquing the LLM's process and solution.
 2. **LLM for Code (capped):** Similar to the above, but for a coding problem using tools like Co-pilot, Cursor, etc.
 3. **Code Cleanup (capped):** Take a homework coding problem and refactor it to reflect best practices in Python and machine learning coding style.
 4. **Problem Modernization:** Update an older homework problem to reflect recent advances and concepts taught in this semester's class.

*The Nature of Deep Learning Deep learning as a field is young. Our understanding is much better than it was a decade ago, but we are still in the infancy of this science. You will often encounter concepts that are not fully understood, which can be uncomfortable.

What is Deep Learning?

A neural network is fundamentally a **simulated, typically analog circuit**.

- It processes information in a massively parallel fashion using continuous values (represented by floating point numbers).
- Its very many parameters are not handcrafted but are set using an **optimization algorithm** (like gradient descent) driven by data.
- It is "deep" because these circuits have many layers; data must pass through many circuit elements to get from input to output.

The goal is for this learned circuit to approximately capture relevant patterns in the training data, so it can be used to solve problems involving new, unseen data.

The "Drunk Man" Principle

A recurring theme in this course can be explained by a joke: *A person sees a drunk man searching for his keys under a streetlamp. "Did you lose your keys here?" the person asks. "No," the drunk man replies, "I lost them over there in the dark." "Then why are you looking here?" "Because this is where the light is."*

Many times in deep learning, we want to solve problem X, but we don't know how. Instead, we solve problem Y, because Y is something we **do** understand and have tools for. It's better to do something with some chance of success than nothing with a guaranteed chance of failure.

The Pile of Linear Algebra

As immortalized by the XKCD comic, much of machine learning involves "stirring a big pile of linear algebra." We lean on linear algebra for several key reasons:

1. **We understand it.** It provides a solid foundation with connections to geometry and intuition.
2. **Computers are good at it.** Linear algebra operations map naturally to parallel hardware like GPUs.
3. It provides a set of powerful building blocks for more complex systems.

Our approach will be to ground the "soup" of deep learning in the elegant and beautiful fundamentals of mathematics you have already learned.

Deep Learning as Modern Alchemy

For most of human history, progress was made empirically, without full theoretical understanding. People built cathedrals before Newton formalized $F = ma$. Alchemists, in their quest to turn lead into gold, developed the experimental apparatus and foundational knowledge that led to modern chemistry.

Deep learning today is in a similar phase. We can build amazing things, but our theoretical understanding often lags behind our empirical results. It's okay to not understand everything fully. This is an opportunity to experience science in a state of formation, much like our ancestors did.

A Brief History of Neural Networks

- **1950s-60s (First Golden Age):** The Perceptron and early data-driven methods like LMS (Least Mean Squares).
- **1969 ("AI Winter"):** The book *Perceptrons* by Minsky and Papert highlighted the limitations of single-layer networks, contributing to a decline in funding and interest.

- **1980s (Second Golden Age):** The popularization of backpropagation for training multi-layer networks led to a resurgence of interest.
- **1990s (Second Winter):** Neural networks were again superseded by methods with stronger mathematical guarantees and better performance at the time (e.g., Support Vector Machines).
- **2000s-2010s (The Rise of GPUs):** The demand for video games drove massive improvements in parallel computing hardware (GPUs). Researchers began using GPUs to train much larger neural networks much faster.
- **2012-Present (Modern Golden Age):** The AlexNet paper demonstrated a massive leap in performance on image classification, decisively beating all previous methods and kicking off the modern deep learning revolution.

Our course will focus primarily on the post-2012 era.

Function Approximation with Neural Networks

Let's build a neural network from first principles. Our goal is to approximate a function $f : \mathbb{R} \rightarrow \mathbb{R}$. We are given training data as pairs (x_i, y_i) , where $y_i \approx f(x_i)$.

Piecewise Approximations

A simple way to approximate a function is with a piecewise function.

- **Piecewise Constant:** Like a Riemann sum, this approximates the function with horizontal steps.
- **Piecewise Linear:** Like the trapezoid rule for integration, this approximates the function with connected line segments. For the same number of "pieces" or "kinks," this is generally a better approximation.

Our goal is to create a neural network that represents a trainable piecewise linear function.

Parameterizing a Piecewise Linear Function

To specify a piecewise linear function, we need to know:

1. The locations of the "kinks" or "elbows" on the x-axis.
2. The slopes of the linear segments between the kinks.
3. An overall vertical offset or intercept (the height at one point).

How can we represent this in a way that is friendly to computation, particularly parallel, linear-algebraic operations? A good approach is to represent the function as a weighted sum of simpler basis functions:

$$N_{\theta}(x) = b + \sum_{i=1}^d w_i g_i(x)$$

This structure is "computer-friendly" because each $g_i(x)$ can be computed in parallel.

The ReLU Basis Function

A powerful choice for the basis function $g_i(x)$ is a function with a single "elbow". By adding many of these functions together with different weights, locations, and slopes, we can construct any arbitrary piecewise linear function.

This elbow function is the **Rectified Linear Unit (ReLU)**, also known as an activation function.

$$\text{ReLU}(z) = \max(0, z)$$

To control the location and slope of the elbow, we make the argument to the ReLU an affine function of the input x :

$$g_i(x) = \text{ReLU}(w'_i x + b'_i)$$

Here, w'_i controls the slope after the kink, and the location of the kink is at $x = -b'_i/w'_i$.

A One-Hidden-Layer Neural Network

Putting it all together, we can represent our piecewise linear approximator as:

$$N_{\theta}(x) = b^2 + \sum_{i=1}^d w_i^2 \cdot \text{ReLU}(w_i^1 x + b_i^1)$$

This is the mathematical form of a **one-hidden-layer multilayer perceptron (MLP)** for a scalar input and scalar output. The parameters θ consist of all the weights and biases $\{W^1, \vec{b}^1, W^2, b^2\}$.

In vectorized form, we can write this as:

$$N_{\theta}(x) = b^2 + W^2 \cdot \text{ReLU}(W^1 x + \vec{b}^1)$$

where W^1 is a matrix (here, a column vector), $\vec{b}^1$ is a vector, W^2 is a matrix (here, a row vector), and b^2 is a scalar. The ReLU function is applied element-wise to the vector argument.

This structure can be visualized as a network diagram:

- **Input Layer:** A single node for the input x .
- **First Hidden Layer:** A set of d nodes. For each node i , the computation $h_i = \text{ReLU}(w_i^1 x + b_i^1)$ is performed. The values h_i are called **activations**.

- **Output Layer:** The activations from the hidden layer are combined in a weighted sum, $b^2 + \sum_i w_i^2 h_i$, to produce the final output.

Lecture 1: Basic Principles

1.1 Recap: Function Approximation with Neural Networks

1.1.1 The Problem: Learning a Function from Samples

We begin by considering the problem of function approximation. Imagine there is a true, unknown function $F : \mathbb{R} \rightarrow \mathbb{R}$. We do not have access to F directly. Instead, we are given a set of training samples, which are pairs of inputs and outputs (x_i, y_i) , where $y_i = F(x_i)$. The goal of machine learning, in this context, is to use these samples to learn an approximation of the true function F .

This can be visualized by plotting the true function (a smooth curve) and then marking the training points (x_i, y_i) that lie on this curve. Our task is to find a function that passes through or near these points, thereby approximating the original unknown function.

1.1.2 Piecewise Linear Approximation

A simple yet powerful way to approximate a continuous function is with a piecewise linear function. Such a function is defined by a series of connected line segments.

A piecewise linear function can be specified by three components:

1. The locations of the "elbows" or kinks.
2. The slopes of the linear segments.
3. An overall vertical bias or offset.

To represent this in a way that is computationally friendly (i.e., "linear-algebra friendly"), we can construct it as a sum of simpler functions. We can use a basis function $g_{\theta}(x)$ that has a single kink. A canonical choice for this is the Rectified Linear Unit (ReLU) function, which is an affine function followed by a $\max(0, \cdot)$ operation.

Specifically, we can define our basis function as:

$$g_{\theta_i}(x) = \max(0, w'_i x + b'_i)$$

By summing up multiple such functions, each with its own parameters (w'_i, b'_i) and a corresponding weight w_i , and adding an overall bias b , we can create a piecewise linear function with as many kinks as we need. The overall approximation $f(x)$ takes the form:

$$f(x) = b + \sum_{i=1}^d w_i g_{\theta_i}(x) = b + \sum_{i=1}^d w_i \max(0, w'_i x + b'_i)$$

1.1.3 Neural Network Representation

The piecewise linear approximation model can be directly translated into the language and structure of a simple neural network.

1.1.4 The ReLU Activation Function

The non-linear function $\text{ReLU}(z) = \max(0, z)$ is a cornerstone of modern deep learning. It is known as the **Rectified Linear Unit** and is a type of **activation function**. These non-linearities are what allow neural networks to approximate complex functions beyond simple linear relationships.

1.1.5 Mathematical and Block Diagram Formulation

We can express our function approximation in a more compact, vectorized form. Let's rename the parameters for clarity, associating them with network layers.

- Let the first layer's weights and biases be W^1 (a $d \times 1$ matrix, where each row is a w'_i) and $\vec{b}^1$ (a $d \times 1$ vector of b'_i).
- Let the second layer's weights and bias be W^2 (a $1 \times d$ matrix of w_i) and b^2 (the scalar bias b).

For a scalar input x , the function becomes:

$$f(x) = b^2 + W^2 \text{ReLU}(W^1 x + \vec{b}^1)$$

Here, the ReLU function is applied element-wise to the vector argument.

This structure can be visualized as a network diagram with distinct layers:

- **Input Layer:** Takes the input x .
- **First Hidden Layer:** Consists of d parallel units. Each unit i computes $h_i = \text{ReLU}(w'_i x + b'_i)$. The outputs of this layer, the vector $\vec{h}$, are called the hidden **activations**.
- **Output Layer:** This is typically an affine layer that combines the hidden activations to produce the final output. In this case, it computes $\sum_i w_i h_i + b^2$.

1.2 Machine Learning vs. Deep Learning Paradigms

1.2.1 Classical Machine Learning

Traditional machine learning is often categorized into two main paradigms.

1.2.2 Supervised Learning

- **Data:** Consists of pairs $(\vec{x}, y)$, where $\vec{x}$ is the input/covariate and y is the output/label.
- **Regression:** The task of predicting a continuous, real-valued output y .
- **Classification:** The task of predicting a discrete label y (e.g., binary or multi-class).
- **General Paradigm:** The common approach is to train a specific model for a specific, well-defined problem using a specific dataset.

1.2.3 Unsupervised Learning

- **Data:** Consists of inputs $\vec{x}$ only, with no corresponding labels y . The goal is to find implicit structure in the data.
- **PCA-style (Dimensionality Reduction):** Finding the directions of maximum variation in the data (e.g., Principal Component Analysis).
- **Clustering:** Grouping similar data points together (e.g., k-means).
- **Density Estimation:** Learning the underlying probability distribution from which the data samples $\vec{x}_1, \vec{x}_2, \dots$ are drawn.

1.2.4 Deep Learning Paradigms

Deep learning extends these classical paradigms and introduces new ones.

1.2.5 Supervised and Unsupervised Deep Learning

Deep networks can be used to solve classical regression and classification problems. They also enable more complex tasks:

- **Localized Annotation (Semantic Segmentation):** A supervised task where the model assigns a label to every pixel in an image, effectively outlining objects (e.g., identifying all pixels corresponding to a cat, a car, or a pedestrian).
- **Learned Embeddings:** An unsupervised task analogous to PCA. The goal is to learn a mapping from complex data (like words or images) into a lower-dimensional vector space, such that similar items are close to each other in this space.
- **Generative Models:** An unsupervised task analogous to density estimation. The goal is to learn a model of the data distribution so that it can generate new, synthetic data samples that resemble the training data (e.g., generating realistic images of cats that have never existed).

1.2.6 Foundation Models

This represents a significant paradigm shift. Instead of training a model from scratch for a specific task, the idea is to:

1. Pre-train a very large model on a massive, general-purpose dataset.
2. This pre-trained model, known as a **foundation model**, learns general representations of the data.
3. This model is then **fine-tuned** on a smaller, task-specific dataset to adapt it to a particular application.

This approach moves away from the "one model, one task" philosophy towards leveraging large-scale, pre-existing knowledge.

1.3 Optimization for Deep Learning

1.3.1 Empirical Risk Minimization (ERM)

Given a dataset of training examples $\{(\vec{x}_i, y_i)\}_{i=1}^n$, our goal is to find the parameters θ of a model $f_\theta(\vec{x})$ that best predict the outputs. We do this by minimizing a loss function $\mathcal{L}$, which measures the discrepancy between the predicted output $f_\theta(\vec{x}_i)$ and the true output y_i .

The standard approach is **Empirical Risk Minimization (ERM)**, where we minimize the average loss over the training data:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_\theta(\vec{x}_i))$$

1.3.2 The True Goal and Key Assumption

The ultimate goal is not just to perform well on the training data, but to generalize to new, unseen data. To formalize this, we make a key assumption: the data pairs $(\vec{x}, y)$ are drawn independently and identically distributed (i.i.d.) from an unknown underlying probability distribution $P(\vec{x}, y)$.

The true objective is to minimize the expected loss, or "risk," over this entire distribution:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\vec{x}, y) \sim P(\vec{x}, y)} [\mathcal{L}(y, f_\theta(\vec{x}))]$$

1.3.3 Challenges in Optimization

Several challenges arise when trying to achieve this goal.

Challenge 1: We don't know the true distribution $P(\vec{x}, y)$. **Solution:** We use a **hold-out test set**. We split our data, use most of it for training, and hold out a portion (the test set) to approximate the true expected loss and evaluate the model's generalization performance.

Challenge 2: The loss function may not be amenable to optimization. For instance, it might be non-differentiable.

Solution: We choose a surrogate loss function that is smooth, continuous, and works well with our chosen optimizer (e.g., gradient descent).

Challenge 3: The model performs well on training data but poorly on test data. This phenomenon is known as **overfitting**. The model has memorized the training examples, including their noise, rather than learning the underlying pattern.

Solution: Regularization.

1.3.4 Regularization and Hyperparameters

Regularization introduces a penalty term to the loss function that discourages overly complex models. A classic example is **Ridge Regression** (L2 regularization), which penalizes large parameter values:

$$\arg \min_{\vec{w}} \|X\vec{w} - \vec{y}\|_2^2 + \lambda \|\vec{w}\|_2^2$$

The term λ is a **hyperparameter** that controls the strength of the regularization.

This introduces a new problem: **hyperparameter search**. Finding the optimal value for λ (and other hyperparameters like learning rate, network architecture, etc.) is critical but can be computationally expensive. Common strategies include:

- **Grid Search / Random Search:** Trying many different hyperparameter values and evaluating their performance on a separate **validation set**. Random search is often more effective than a fixed grid.
- **Couple Hyperparameters:** Scale related hyperparameters together to reduce the dimensionality of the search space.
- **Use What Others Did:** Start with hyperparameter values that have been reported to work well in similar research papers or applications.

1.4 Gradient Descent

1.4.1 The Algorithm

Gradient descent is the workhorse optimization algorithm for deep learning. It is an iterative method for finding a local minimum of a differentiable function. Given a loss function $\mathcal{L}(\theta)$ (here, we abbreviate the dependency on training data), the update rule is:

1. Start with an initial guess for the parameters, θ_0 .
2. For each step $t = 0, 1, 2, \dots$, update the parameters by moving in the direction of the negative gradient:

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \eta \nabla_{\theta} \mathcal{L}(\vec{\theta}_t)$$

where η is the **learning rate**, a hyperparameter that controls the step size.

1.4.2 Analysis for the Least Squares Problem

Let's analyze gradient descent for the simple case of least squares, where the loss is $\mathcal{L}(\vec{w}) = \|X\vec{w} - \vec{y}\|_2^2$. The optimal solution is $\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$.

The gradient of the loss function is:

$$\nabla_{\vec{w}} \mathcal{L}(\vec{w}) = 2X^T(X\vec{w} - \vec{y})$$

The gradient descent update rule becomes:

$$\begin{aligned}\vec{w}_{t+1} &= \vec{w}_t - \eta (2X^T(X\vec{w}_t - \vec{y})) \\ &= (I - 2\eta X^T X)\vec{w}_t + 2\eta X^T \vec{y}\end{aligned}$$

1.4.3 Convergence and Optimal Learning Rate

To analyze convergence, we look at the error vector $\vec{e}_t = \vec{w}_t - \vec{w}^*$. The update rule for the error is:

$$\vec{w}_{t+1} - \vec{w}^* = (I - 2\eta X^T X)(\vec{w}_t - \vec{w}^*)$$

The algorithm converges if the error vector shrinks to zero. This happens if all eigenvalues of the matrix $(I - 2\eta X^T X)$ have an absolute value less than 1.

Let λ_i be an eigenvalue of the symmetric positive semi-definite matrix $X^T X$. The corresponding eigenvalue of the update matrix is $1 - 2\eta\lambda_i$. We require:

$$|1 - 2\eta\lambda_i| < 1 \quad \text{for all } i$$

This inequality must hold for both the smallest eigenvalue, $\lambda_{\min}$, and the largest, $\lambda_{\max}$. The tightest constraint comes from $\lambda_{\max}$:

$$-1 < 1 - 2\eta\lambda_{\max} \implies 2\eta\lambda_{\max} < 2 \implies \eta < \frac{1}{\lambda_{\max}}$$

For the **fastest convergence**, we want to make the largest absolute eigenvalue as small as possible. This occurs when the bounds for $\lambda_{\min}$ and $\lambda_{\max}$ are balanced:

$$|1 - 2\eta\lambda_{\max}| = |1 - 2\eta\lambda_{\min}|$$

Solving this gives the optimal learning rate:

$$\eta^* = \frac{1}{\lambda_{\min} + \lambda_{\max}}$$

The rate of convergence is then determined by $\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}}$, which is closely related to the **condition number** $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ of the matrix $X^T X$. A high condition number leads to slow convergence.

1.5 Regularization and Gradient Descent

1.5.1 Ridge Regression and SVD

The solution to Ridge Regression is given by:

$$\vec{w}^* = (X^T X + \lambda I)^{-1} X^T \vec{y}$$

An alternative form, sometimes called the kernel form, is $\vec{w}^* = X^T (X X^T + \lambda I)^{-1} \vec{y}$. This can be more efficient if the number of data points is smaller than the number of features.

To gain insight, we can use the Singular Value Decomposition (SVD) of the data matrix $X = U \Sigma V^T$. Substituting this into the Ridge solution and simplifying, we get:

$$\vec{w}^* = V (\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T U^T \vec{y}$$

This can be expressed as a sum over the singular components:

$$\vec{w}^* = \sum_{i=1}^k \vec{v}_i \frac{\sigma_i}{\sigma_i^2 + \lambda} (\vec{u}_i^T \vec{y})$$

where $\vec{v}_i$ and $\vec{u}_i$ are the right and left singular vectors and σ_i are the singular values.

This form reveals how Ridge regularization works:

- For a large singular value $\sigma_i \gg \sqrt{\lambda}$, the term $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx \frac{1}{\sigma_i}$. The component is kept.
- For a small singular value $\sigma_i \ll \sqrt{\lambda}$, the term $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx \frac{\sigma_i}{\lambda}$, which is small. The component is heavily shrunk towards zero.

Regularization effectively filters out the directions in the data corresponding to small singular values, which are often associated with noise.

This connects to the idea of the **implicit regularization of gradient descent**. As will be discussed later, running gradient descent for a limited number of steps (early stopping) has an effect similar to explicit L2 regularization, where the number of steps plays a role analogous to the inverse of the regularization parameter λ .

Lecture 2: Optimization: implicit regularization, SGD, and momentum

2.1 Implicit Regularization of Gradient Descent

2.1.1 Recap: Convergence of Gradient Descent for Linear Regression

Last time, we analyzed the convergence of Gradient Descent (GD) for linear regression. A key result was the condition on the learning rate η :

$$\eta < \frac{1}{\lambda_{\max}(X^T X)} = \frac{1}{\sigma_{\max}(X)^2} \quad (1)$$

For GD to converge, the learning rate must be smaller than the inverse of the largest eigenvalue of the covariance matrix $X^T X$.

The main takeaway is that the **geometric properties of the loss function** are crucial. They dictate the practical choices for hyperparameters like the learning rate. Concepts like smoothness and strong convexity are generalizations of these properties for non-quadratic functions, where we can bound the function from above and below by quadratics. The convergence rate then often depends on the ratio of these bounds, analogous to how the convergence of GD for least squares depends on the condition number $\frac{\sigma_{\max}}{\sigma_{\min}}$.

2.1.2 Ridge Regression in SVD Coordinates

To understand the connection between optimization and regularization, let's analyze Ridge Regression. The solution for Ridge is given by:

$$\vec{w}^* = (X^T X + \lambda I)^{-1} X^T \vec{y} \quad (2)$$

This is the standard form. There is also a "kernel form":

$$\vec{w}^* = X^T (X X^T + \lambda I)^{-1} \vec{y} \quad (3)$$

Let's analyze the standard form in the basis defined by the Singular Value Decomposition (SVD) of the data matrix X . Let $X = U \Sigma V^T$. Substituting this into the solution:

$$\vec{w}^* = ((U \Sigma V^T)^T (U \Sigma V^T) + \lambda I)^{-1} (U \Sigma V^T)^T \vec{y} \quad (4)$$

$$= (V \Sigma^T U^T U \Sigma V^T + \lambda I)^{-1} V \Sigma^T U^T \vec{y} \quad (5)$$

$$= (V \Sigma^T \Sigma V^T + \lambda V V^T)^{-1} V \Sigma^T U^T \vec{y} \quad (\text{since } U^T U = I \text{ and } I = V V^T) \quad (6)$$

$$= (V (\Sigma^T \Sigma + \lambda I) V^T)^{-1} V \Sigma^T U^T \vec{y} \quad (7)$$

$$= (V^T)^{-1} (\Sigma^T \Sigma + \lambda I)^{-1} V^{-1} V \Sigma^T U^T \vec{y} \quad (\text{using } (ABC)^{-1} = C^{-1} B^{-1} A^{-1}) \quad (8)$$

$$= V (\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T U^T \vec{y} \quad (\text{since } V^{-1} = V^T \text{ and } (V^T)^{-1} = V) \quad (9)$$

The matrix $(\Sigma^T \Sigma + \lambda I)$ is diagonal. Its diagonal entries are $\sigma_i^2 + \lambda$ for the non-zero singular values and λ for the rest. The inverse is also diagonal with entries $1/(\sigma_i^2 + \lambda)$.

We can express the solution $\vec{w}^*$ as a linear combination of the right singular vectors $\vec{v}_i$:

$$\vec{w}^* = \sum_{i=1}^{\text{rank}(X)} \vec{v}_i \left(\frac{\sigma_i}{\sigma_i^2 + \lambda} \right) (\vec{u}_i^T \vec{y}) \quad (10)$$

The term $\vec{u}_i^T \vec{y}$ is a scalar projection. The key is the scaling factor $\frac{\sigma_i}{\sigma_i^2 + \lambda}$.

- If λ is very large compared to σ_i^2 (i.e., for small singular values), the scaling factor approaches 0:

$$\lambda \gg \sigma_i^2 \implies \frac{\sigma_i}{\sigma_i^2 + \lambda} \rightarrow 0$$

This means the components of the solution corresponding to small singular values are heavily suppressed.

- If σ_i^2 is very large compared to λ , the scaling factor approaches $1/\sigma_i$:

$$\sigma_i^2 \gg \lambda \implies \frac{\sigma_i}{\sigma_i^2 + \lambda} \approx \frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$$

Intuition: Small singular values correspond to directions of low variance in the data. Ridge regularization prevents the model from fitting patterns in these low-variance directions, which are often dominated by noise. This helps prevent overfitting and improves generalization.

2.1.3 Implicit Regularization via Gradient Descent and Early Stopping

Let's analyze the standard GD update for least squares, but this time in the SVD basis. The update rule is: $\vec{w}_{t+1} = \vec{w}_t - 2\eta X^T(X\vec{w}_t - \vec{y})$. Let's change to the basis defined by V by defining $\tilde{\vec{w}}_t = V^T \vec{w}_t$.

$$V^T \vec{w}_{t+1} = V^T \vec{w}_t - 2\eta V^T (V \Sigma^T U^T) (U \Sigma V^T \vec{w}_t - \vec{y}) \quad (11)$$

$$\tilde{\vec{w}}_{t+1} = \tilde{\vec{w}}_t - 2\eta \Sigma^T U^T (U \Sigma \tilde{\vec{w}}_t - \vec{y}) \quad (12)$$

$$\tilde{\vec{w}}_{t+1} = \tilde{\vec{w}}_t - 2\eta (\Sigma^T \Sigma \tilde{\vec{w}}_t - \Sigma^T U^T \vec{y}) \quad (13)$$

Since $\Sigma^T \Sigma$ is a diagonal matrix, this system of equations is decoupled. We can look at the update for each component i individually:

$$\tilde{w}_{t+1,i} = \tilde{w}_{t,i} - 2\eta (\sigma_i^2 \tilde{w}_{t,i} - \sigma_i \tilde{y}_i) \quad (14)$$

where $\tilde{y}_i$ is the i -th component of $U^T \vec{y}$. The magnitude of the gradient step in the direction of the i -th singular vector is proportional to σ_i^2 .

- **Large singular values (σ_i):** GD takes large steps and converges quickly in these directions.

- **Small singular values (σ_i):** GD takes very small steps and converges slowly in these directions.

This leads to the concept of **Early Stopping**. If we run GD but stop it before it fully converges, the weights will have moved significantly in the directions of large singular values but very little in the directions of small singular values.

- **Ridge Regression:** Explicitly suppresses components related to small singular values via the λ parameter.
- **GD with Early Stopping:** Implicitly suppresses these same components because it doesn't have enough time to converge in those slow-moving directions.

Thus, early stopping acts as a form of implicit regularization, achieving a similar effect to Ridge without an explicit penalty term.

Weight Decay The GD update for Ridge regression is:

$$\vec{w}_{t+1} = \vec{w}_t - 2\eta(X^T(X\vec{w}_t - \vec{y}) + \lambda\vec{w}_t) = (1 - 2\eta\lambda)\vec{w}_t - 2\eta X^T(X\vec{w}_t - \vec{y})$$

The term $(1 - 2\eta\lambda)$ multiplicatively shrinks the weights at each step. This is known as **weight decay**. In practice, sometimes even without an explicit Ridge penalty in the loss, a weight decay term is added directly to the optimizer's update rule to prevent weights from growing too large.

2.2 Stochastic Gradient Descent (SGD)

While GD is effective for convex problems, training deep neural networks involves optimizing highly non-convex loss landscapes over massive datasets.

2.2.1 The SGD Algorithm

Many loss functions can be decomposed as a sum over the individual data points.

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta) \quad (15)$$

where $f_i(\theta)$ is the loss on the i -th data point. Computing the full gradient $\nabla f(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\theta)$ requires a full pass over the dataset, which is computationally expensive if n is large.

SGD approximates the full gradient using a small, randomly sampled subset of the data called a **mini-batch**. The SGD update rule is:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla f_i(\theta_t) \quad (16)$$

where $\mathcal{B}_t$ is a mini-batch of size B sampled uniformly at random from the dataset at step t .

- **Mini-batch SGD:** Standard approach with $1 < B \ll n$.
- **Full-batch SGD:** This is just standard Gradient Descent ($B = n$).

A key property is that the mini-batch gradient is an **unbiased estimator** of the true gradient:

$$\mathbb{E}_{i \sim \text{Uniform}}[\nabla f_i(\theta)] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\theta) = \nabla f(\theta) \quad (17)$$

On average, SGD moves in the correct direction.

2.2.2 Properties of SGD

- **Advantages:**
 1. **Computationally efficient:** Requires processing only a small batch of data per update.
 2. **Escaping local minima:** The noise in the gradient estimate can help the optimizer “jump out” of sharp local minima or move past saddle points, potentially finding a better, more generalizable solution.
- **Disadvantages:**
 1. **Noisy updates:** The convergence path is much more erratic than GD.
 2. **Convergence guarantees:** Traditional optimization theory requires a decaying learning rate schedule (e.g., $\eta_t \propto 1/t$) for SGD to provably converge. However, in deep learning, SGD with a constant step size is often used and can still converge, particularly in over-parameterized regimes.

2.3 Momentum

2.3.1 Motivation: The Problem of Oscillation

In GD, the learning rate η is limited by the largest singular value, $\sigma_{\max}$. This choice can lead to slow convergence along directions with smaller singular values. If we try to increase η to speed up convergence, we can introduce oscillations, where the optimizer bounces back and forth across a ravine in the loss landscape instead of moving smoothly along the bottom.

How can we accelerate convergence while damping these oscillations? The key idea is to average gradients over time.

2.3.2 An Exponential Moving Average of Gradients

Instead of a simple moving average, which requires storing many past gradients, we can use a more memory-efficient **exponential moving average**. This idea is analogous to a low-pass filter in an RC circuit, which smooths out high-frequency signals by maintaining a state (the charge on the capacitor).

The Momentum algorithm maintains a "velocity" vector $\vec{z}_k$, which is an exponential moving average of past gradients. The update rules are:

$$\vec{z}_{k+1} = \beta \vec{z}_k + (1 - \beta) \nabla f(\vec{w}_k) \quad (\text{with } \vec{z}_0 = \mathbf{0}) \quad (18)$$

$$\vec{w}_{k+1} = \vec{w}_k - \eta \vec{z}_{k+1} \quad (19)$$

Here, $\beta \in [0, 1)$ is the momentum hyperparameter. A typical value is $\beta = 0.9$. If we expand the recurrence for $\vec{z}_k$, we can see its structure:

$$\vec{z}_1 = (1 - \beta) \nabla f(\vec{w}_0)$$

$$\vec{z}_2 = \beta(1 - \beta) \nabla f(\vec{w}_0) + (1 - \beta) \nabla f(\vec{w}_1)$$

$$\vec{z}_3 = \beta^2(1 - \beta) \nabla f(\vec{w}_0) + \beta(1 - \beta) \nabla f(\vec{w}_1) + (1 - \beta) \nabla f(\vec{w}_2)$$

The velocity $\vec{z}_k$ is a weighted sum of all past gradients, with weights that decay exponentially into the past.

Intuition: Momentum acts like a ball rolling down a hill.

- In directions where the gradient is consistently pointing the same way, the velocity builds up, and the optimizer accelerates.
- In directions where the gradient oscillates, the velocity averages out these oscillations, leading to smaller steps and a smoother path.

The main benefit is **faster convergence**. Theoretically, for quadratic objectives, momentum changes the dependency of the convergence rate from the condition number κ to $\sqrt{\kappa}$, which is a significant improvement.

2.4 A Look at SGD Convergence (Underdetermined Case)

We can prove that for certain problems, SGD can converge with a constant step size. Let's analyze the case of an underdetermined linear system where a zero-loss solution exists.

2.4.1 Problem Setup

Consider the system $X\vec{w} = \vec{y}$, where $X \in \mathbb{R}^{n \times d}$ with $d > n$ (a wide matrix) and X has full row rank. There are infinitely many solutions. We are interested in the minimum-norm solution:

$$\vec{w}^* = X^T (X X^T)^{-1} \vec{y} \quad (20)$$

We analyze SGD with batch size $B = 1$, initialized at $\vec{w}_0 = \mathbf{0}$.

2.4.2 Change of Variables and Simplification

To analyze convergence to $\vec{w}^*$, we study the evolution of the error vector, $\vec{q}_t = \vec{w}_t - \vec{w}^*$. Since $X\vec{w}_t - \vec{y}$ is the residual and $X\vec{w}^* = \vec{y}$, the problem is equivalent to finding a solution to $X\vec{q} = \mathbf{0}$, with initial error $\vec{q}_0 = -\vec{w}^*$.

Using the SVD $X = U[\Sigma_1 \ 0]V^T$, we can show that the dynamics of the system are confined to the first n dimensions of the V basis. This allows us to reduce the problem to an equivalent, square system $\tilde{X}\vec{z} = \mathbf{0}$, where $\vec{z} \in \mathbb{R}^n$ represents the coordinates of the error in the relevant subspace.

2.4.3 SGD Update and Proof Strategy

For this simplified system, the GD update is $\vec{z}_{t+1} = \vec{z}_t - 2\eta\tilde{X}^T\tilde{X}\vec{z}_t$. The SGD update (with batch size 1) is:

$$\vec{z}_{t+1} = \vec{z}_t - 2\eta\tilde{X}_{I_t}\tilde{X}_{I_t}^T\vec{z}_t \quad (21)$$

where I_t is the index of the data point randomly sampled at step t .

Our goal is to show that the loss, $L(\vec{z}_t) = \|\tilde{X}\vec{z}_t\|^2$, goes to zero. The proof strategy relies on a **Lyapunov function** argument. We want to show that the expected value of the loss decreases at every step, i.e., it is a supermartingale. If we can show that

$$\mathbb{E}[L(\vec{z}_{t+1})|\vec{z}_t] \leq (1 - c)L(\vec{z}_t) \quad (22)$$

for some positive constant c , then since the loss is non-negative, it must converge to zero. The derivation involves analyzing the expectation of the change in loss, which depends on properties of the data matrix and the learning rate.

Lecture 3: Optimization: Adam, locally linear perspective, features

3.1 Convergence of SGD with Constant Step Size

We continue the discussion on Stochastic Gradient Descent (SGD) from the previous lecture. While SGD has computational advantages and can escape local minima, its noisy nature typically requires a decaying step size for convergence proofs. Here, we analyze a special case where SGD with a **constant step size** is proven to converge.

3.1.1 Problem Setup

We consider an underdetermined linear system of equations:

$$\mathbf{X}\vec{w} = \vec{y} \quad (23)$$

where $\mathbf{X} \in \mathbb{R}^{n \times d}$ is a wide matrix ($d > n$) with full row rank, $\vec{w} \in \mathbb{R}^d$ are the weights, and $\vec{y} \in \mathbb{R}^n$ are the labels. Since the system is underdetermined, there are infinitely many solutions. We are interested in the **minimum norm solution**, $\vec{w}^*$:

$$\vec{w}^* = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \vec{y} \quad (24)$$

We analyze SGD with a batch size of 1, a constant learning rate η , and an initialization of $\vec{w}_0 = \vec{0}$.

3.1.2 Change of Coordinates

To analyze convergence to the optimum $\vec{w}^*$, it's more convenient to work in the **error space**. We define a new coordinate system based on the deviation from the optimal solution:

$$\vec{q}_t = \vec{w}_t - \vec{w}^* \quad (25)$$

Since $\mathbf{X}\vec{w}_t = \vec{y}$ and $\mathbf{X}\vec{w}^* = \vec{y}$ at the solution, our system of equations in the new coordinates becomes:

$$\mathbf{X}\vec{q} = \vec{0} \quad (26)$$

The initial condition also transforms:

$$\vec{q}_0 = \vec{w}_0 - \vec{w}^* = -\vec{w}^* \quad (27)$$

3.1.3 Singular Value Decomposition (SVD) Basis

To simplify the analysis, we perform another change of basis using the SVD of $\mathbf{X}$:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \quad (28)$$

Since $\mathbf{X}$ is a wide matrix ($n \times d$), the singular value matrix $\mathbf{\Sigma}$ has the structure:

$$\mathbf{\Sigma} = [\mathbf{\Sigma}_1 | \mathbf{0}], \quad \text{where } \mathbf{\Sigma}_1 \in \mathbb{R}^{n \times n} \text{ is a diagonal matrix of singular values.} \quad (29)$$

We define a new set of coordinates $\vec{z}$ by projecting $\vec{q}$ onto the right singular vector basis:

$$\vec{z} = \mathbf{V}^\top \vec{q} \quad (30)$$

The system $\mathbf{X}\vec{q} = \vec{0}$ becomes $\mathbf{U}[\mathbf{\Sigma}_1 | \mathbf{0}]\mathbf{V}^\top \vec{q} = \vec{0}$, which simplifies to $\mathbf{U}[\mathbf{\Sigma}_1 | \mathbf{0}]\vec{z} = \vec{0}$. Let $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}_1$. If we partition $\vec{z}$ into its first n components ($\tilde{z}$) and its last $d - n$ components, the equation becomes $\tilde{\mathbf{X}}\tilde{z} = \vec{0}$.

Crucially, the initial condition $\vec{z}_0 = \mathbf{V}^\top \vec{q}_0 = -\mathbf{V}^\top \vec{w}^*$ can be shown to have zero entries for its last $d - n$ components. This means the dynamics of the optimization remain confined to the first n coordinates, and we can analyze the simpler system $\tilde{\mathbf{X}}\tilde{z} = \vec{0}$ where $\tilde{\mathbf{X}}$ is an invertible $n \times n$ matrix.

3.1.4 SGD Update and Convergence Proof

We consider the squared loss function:

$$\mathcal{L}(\vec{z}) = \|\tilde{\mathbf{X}}\vec{z}\|_2^2 \quad (31)$$

The gradient with respect to $\vec{z}$ is $\nabla \mathcal{L}(\vec{z}) = 2\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\vec{z}$. For SGD with batch size 1, we randomly select a data point (row) i_t at each step t . The update rule in the $\vec{z}$ -coordinates is:

$$\vec{z}_{t+1} = \vec{z}_t - 2\eta \tilde{x}_{i_t} (\tilde{x}_{i_t}^\top \vec{z}_t) \quad (32)$$

where $\tilde{x}_{i_t}^\top$ is the i_t -th row of $\tilde{\mathbf{X}}$.

To prove convergence, we use a **Lyapunov function** argument. We show that the expected value of the loss at step $t + 1$, conditioned on the state at step t , is strictly decreasing.

1. **Expand the loss at $t + 1$:** We can write $\mathcal{L}(\vec{z}_{t+1})$ in terms of $\mathcal{L}(\vec{z}_t)$:

$$\begin{aligned} \mathcal{L}(\vec{z}_{t+1}) &= \|\tilde{\mathbf{X}}(\vec{z}_t + (\vec{z}_{t+1} - \vec{z}_t))\|_2^2 \\ &= \underbrace{\|\tilde{\mathbf{X}}\vec{z}_t\|_2^2}_{\mathcal{L}(\vec{z}_t)} + \underbrace{2\vec{z}_t^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}(\vec{z}_{t+1} - \vec{z}_t)}_A + \underbrace{\|\tilde{\mathbf{X}}(\vec{z}_{t+1} - \vec{z}_t)\|_2^2}_B \end{aligned}$$

2. **Analyze the expected value of term A:** The randomness comes from the choice of data point i_t . The expectation is taken over this choice.

$$\begin{aligned} \mathbb{E}[A | \vec{z}_t] &= 2\vec{z}_t^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \mathbb{E}[\vec{z}_{t+1} - \vec{z}_t | \vec{z}_t] \\ &= 2\vec{z}_t^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \mathbb{E}[-2\eta \tilde{x}_{i_t} (\tilde{x}_{i_t}^\top \vec{z}_t) | \vec{z}_t] \\ &= -4\eta \vec{z}_t^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \left(\mathbb{E}[\tilde{x}_{i_t} \tilde{x}_{i_t}^\top] \right) \vec{z}_t \end{aligned}$$

Since we sample uniformly, $\mathbb{E}[\tilde{x}_{i_t}\tilde{x}_{i_t}^\top] = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i\tilde{x}_i^\top = \frac{1}{n} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$.

$$\mathbb{E}[A|\vec{z}_t] = -\frac{4\eta}{n} \vec{z}_t^\top (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^2 \vec{z}_t = -\frac{4\eta}{n} \|\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \vec{z}_t\|_2^2 \quad (33)$$

Using the minimum singular value of $\tilde{\mathbf{X}}$, $\sigma_{\min}$, we can bound this term:

$$\mathbb{E}[A|\vec{z}_t] \leq -\frac{4\eta}{n} \sigma_{\min}^2 (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}) \|\vec{z}_t\|_2^2 \leq -\frac{4\eta}{n} \sigma_{\min}^2 (\tilde{\mathbf{X}}) \|\tilde{\mathbf{X}} \vec{z}_t\|_2^2 = -\frac{4\eta}{n} \sigma_{\min}^2 (\tilde{\mathbf{X}}) \mathcal{L}(\vec{z}_t) \quad (34)$$

3. **Bound term B:** Term B is quadratic in the update and is always non-negative. We can show that its expectation is bounded by a term proportional to $\mathcal{L}(\vec{z}_t)$:

$$\mathbb{E}[B|\vec{z}_t] \leq C_1 \eta^2 \sigma_{\max}^2 \max_i (\|\tilde{x}_i\|^2) \mathcal{L}(\vec{z}_t) \quad (35)$$

where C_1 is a constant. The key insight is that this term depends on the learning rate squared (η^2) and the norms of the data points.

4. **Combine terms:** Putting everything together:

$$\mathbb{E}[\mathcal{L}(\vec{z}_{t+1})|\vec{z}_t] \leq \left(1 - \frac{4\eta}{n} \sigma_{\min}^2 + C_1 \eta^2 \sigma_{\max}^2 \max_i (\|\tilde{x}_i\|^2)\right) \mathcal{L}(\vec{z}_t) \quad (36)$$

By choosing a sufficiently small constant step size η , we can ensure that the factor multiplying $\mathcal{L}(\vec{z}_t)$ is less than 1. This guarantees that the expected loss decreases at every step, leading to convergence to $\mathcal{L}(\vec{z}) = 0$.

Takeaways:

- SGD can converge with a constant step size in certain well-behaved settings.
- The convergence rate depends on the spectral properties of the data matrix (singular values).
- The norms of the individual data points matter, highlighting the importance of data normalization.

3.2 The Adam Optimizer

3.2.1 Motivation: The Challenge of Gradient Descent

Gradient Descent (GD) exhibits a behavior that is both a feature and a bug: it takes larger steps in directions corresponding to large singular values and smaller steps in directions corresponding to small singular values.

- **Advantage:** This behavior leads to **implicit regularization**. By using early stopping, GD tends to find solutions that are simple in the basis of the top singular vectors, often corresponding to low-norm solutions that generalize well.

- **Disadvantage:** It is difficult to choose a single learning rate η . If η is too large, the optimization can become unstable and diverge along the directions of large singular values. If η is too small, progress along directions of small singular values becomes extremely slow, leading to long convergence times.

An ideal (but computationally infeasible) solution would be to compute the SVD of the data matrix and use a different, normalized learning rate for each singular direction.

3.2.2 Normalizing Gradients: Sign SGD

A practical approach is to normalize the gradient updates in the standard coordinate basis. Instead of using the gradient directly, we can use its sign. This ensures that the step size is uniform in every coordinate direction.

$$\vec{w}_{t+1} = \vec{w}_t - \eta \cdot \text{sgn}(\nabla_{\vec{w}} \mathcal{L}(\vec{w}_t)) \quad (37)$$

This can be implemented by dividing the gradient by the square root of its element-wise square:

$$\vec{w}_{t+1} = \vec{w}_t - \eta \frac{\nabla_{\vec{w}} \mathcal{L}(\vec{w}_t)}{\sqrt{(\nabla_{\vec{w}} \mathcal{L}(\vec{w}_t))^2 + \epsilon}} \quad (38)$$

where the square is element-wise, and ϵ is a small constant for numerical stability. **Problem:** This "Sign SGD" method can be too aggressive. Near the optimum, the gradient magnitude becomes small, but the sign update remains large, causing the algorithm to overshoot and oscillate around the minimum without converging.

3.2.3 Adam: Adaptive Moment Estimation

Adam (Adaptive Moment Estimation) refines this idea by smoothing the normalization process using momentum. It maintains two exponentially decaying moving averages:

1. **First Moment (Mean):** An estimate of the mean of the gradients (like in momentum).
2. **Second Moment (Uncentered Variance):** An estimate of the mean of the element-wise squared gradients.

Let $\vec{g}_t = \nabla_{\vec{w}} \mathcal{L}(\vec{w}_t)$. The moving averages are updated as follows:

$$\vec{m}_t = \beta_1 \vec{m}_{t-1} + (1 - \beta_1) \vec{g}_t \quad (39)$$

$$\vec{v}_t = \beta_2 \vec{v}_{t-1} + (1 - \beta_2) (\vec{g}_t \odot \vec{g}_t) \quad (40)$$

where β_1 and β_2 are decay rates (e.g., 0.9 and 0.999), and $\odot$ denotes element-wise multiplication.

Bias Correction: Since $\vec{m}_0$ and $\vec{v}_0$ are initialized to zero, the estimates $\vec{m}_t$ and $\vec{v}_t$ are biased

towards zero, especially during the initial steps. Adam corrects for this bias:

$$\hat{\vec{m}}_t = \frac{\vec{m}_t}{1 - \beta_1^t} \quad (41)$$

$$\hat{\vec{v}}_t = \frac{\vec{v}_t}{1 - \beta_2^t} \quad (42)$$

The final Adam update rule combines these components:

$$\vec{w}_{t+1} = \vec{w}_t - \eta \frac{\hat{\vec{m}}_t}{\sqrt{\hat{\vec{v}}_t + \epsilon}} \quad (43)$$

3.2.4 Properties of Adam

- **Vanishing/Exploding Gradients:** The normalization inherent in Adam helps mitigate the issues of vanishing or exploding gradients in deep networks. The step size is less dependent on the gradient's magnitude, making it more stable.
- **Memory Cost:** Adam requires more memory than simpler optimizers.
 - SGD: Stores only the parameters (1x memory).
 - SGD with Momentum: Stores parameters and the first moment vector (2x memory).
 - Adam: Stores parameters, first moment vector, and second moment vector (3x memory).

3.2.5 AdamW: Adam with Decoupled Weight Decay

Standard L2 regularization (weight decay) is implemented by adding $\frac{\lambda}{2} \|\vec{w}\|^2$ to the loss. In GD, this results in an update term that shrinks the weights: $\vec{w}_{t+1} = (1 - \eta\lambda)\vec{w}_t - \eta\nabla\mathcal{L}_{\text{data}}$.

This approach does not work well with Adam. The gradient from the L2 penalty ($\lambda\vec{w}$) gets included in the adaptive normalization. This couples the effect of the regularization strength λ with the adaptive learning rate, leading to suboptimal performance.

AdamW solves this by **decoupling** the weight decay from the gradient-based update. The Adam update is performed using only the data loss gradient, and then the weight decay is applied directly to the weights.

$$\vec{w}_{t+1} = (1 - \lambda') \left(\vec{w}_t - \eta \frac{\hat{\vec{m}}_t}{\sqrt{\hat{\vec{v}}_t + \epsilon}} \right) \quad (44)$$

where λ' is the weight decay rate. This makes the regularization more effective and predictable.

3.3 Neural Networks as Linearized Models

To understand how optimizers behave on the complex, non-convex loss landscapes of neural networks, it is useful to adopt a simplified, linear perspective.

3.3.1 Perspective 1: Feature Extraction

A deep neural network can be viewed as a two-stage model:

1. **Feature Extractor:** The initial non-linear layers of the network, $h(\vec{x})$, transform the input data $\vec{x}$ into a high-level feature representation.
2. **Linear Model:** The final layer of the network acts as a simple linear model (e.g., a classifier or regressor) that operates on these learned features.

The key idea is that the network *learns* the optimal features from the data, rather than having them pre-specified.

This perspective gives rise to the practical technique of **linear probing**. For a new task, one can take a pre-trained network, freeze the weights of the feature extraction layers, and only re-train the final linear layer. This is an efficient way to adapt a powerful model to new data.

3.3.2 Perspective 2: Taylor Expansion

Another way to linearize a network is to consider its first-order Taylor expansion with respect to its parameters, $\vec{\theta}$. Let the network function be $f(\vec{x}, \vec{\theta})$. If we take a small step from $\vec{\theta}_0$ to $\vec{\theta}_0 + \Delta\vec{\theta}$, the network's output changes approximately as:

$$f(\vec{x}, \vec{\theta}_0 + \Delta\vec{\theta}) \approx f(\vec{x}, \vec{\theta}_0) + \langle \nabla_{\vec{\theta}} f(\vec{x}, \vec{\theta}_0), \Delta\vec{\theta} \rangle \quad (45)$$

This is a linear model in the parameter update $\Delta\vec{\theta}$. The "features" of this model are the components of the gradient of the network's output with respect to its parameters, $\nabla_{\vec{\theta}} f(\vec{x}, \vec{\theta}_0)$.

This linear approximation is valid under the **lazy training assumption**, which posits that the parameters do not move far from their initialization during training (i.e., $\|\Delta\vec{\theta}\|$ is small). This regime is often observed in very wide, over-parameterized networks. This perspective is the foundation for the theory of the **Neural Tangent Kernel (NTK)**, which we will explore in future lectures.

Lecture 4: Optimizers

4.1 Linear Perspectives on Neural Networks

We can analyze neural networks through a linear lens in two primary ways. The first perspective, discussed previously, treats the network as a feature extractor followed by a final linear layer.

4.1.1 A Second Perspective: Linearization via Taylor Expansion

A second linear perspective involves analyzing the network as a function of its parameters. Let a neural network be represented by a function $f(x, \theta)$, where x is the input data and $\theta \in \mathbb{R}^m$ are the network parameters (weights and biases).

We can analyze how the network's output changes with a small perturbation $\Delta\theta$ to the parameters using a Taylor expansion around the current parameters θ :

$$f(x, \theta + \Delta\theta) = f(x, \theta) + \langle \nabla_{\theta} f(x, \theta), \Delta\theta \rangle + \frac{1}{2} \Delta\theta^T H \Delta\theta + \dots \quad (46)$$

where $\nabla_{\theta} f$ is the gradient of the function with respect to its parameters, and H is the Hessian matrix of second derivatives.

By ignoring the higher-order terms, we obtain a first-order or linear approximation of the network's behavior:

$$f(x, \theta + \Delta\theta) \approx f(x, \theta) + \langle \nabla_{\theta} f(x, \theta), \Delta\theta \rangle \quad (47)$$

The Lazy Training Assumption This linear approximation is only valid when the change in parameters, $\Delta\theta$, is small. The **lazy training assumption** posits that during training, the network's weights move by very small amounts. The final trained parameters θ_{final} remain very close to the initial parameters θ_{initial} . This assumption, often observed in practice for very wide networks, justifies the use of this linear model.

Gradients as Features In the linear term $\langle \nabla_{\theta} f, \Delta\theta \rangle$, the components of the gradient vector $\nabla_{\theta} f$ act as coefficients that multiply the changes in parameters $\Delta\theta_j$. By analogy to standard linear models ($y = Xw$), we can think of the entries of the gradient, $\frac{\partial f}{\partial \theta_j}$, as the **features** of the network with respect to its parameters. This perspective is foundational to the theory of the **Neural Tangent Kernel (NTK)**, which analyzes the training dynamics of infinitely wide neural networks.

4.2 Revisiting Descent Algorithms

This linear perspective provides a new lens through which to view optimization algorithms like Adam and Gradient Descent.

4.2.1 The Core Tension in Optimization

When training a model, a fundamental tension exists:

- We want to take **large steps** to converge to a minimum quickly.
- We must take **small steps** to ensure the local linear approximation of the loss landscape, upon which gradient-based methods rely, remains valid.

Taking a step that is too large can invalidate the gradient information, potentially leading to an increase in the loss.

4.2.2 Optimization as a Constrained Problem

A principled way to resolve this tension is to frame the step selection as a constrained optimization problem. We seek to find the step $\Delta\theta$ that maximally decreases the linearized loss function, subject to a constraint on the size of the step.

$$\Delta\theta^* = \arg \min_{\|\Delta\theta\| \leq \eta} \mathcal{L}(\theta_i + \Delta\theta) \quad (48)$$

Using the first-order approximation for the loss $\mathcal{L}$, this becomes:

$$\Delta\theta^* \approx \arg \min_{\|\Delta\theta\| \leq \eta} \mathcal{L}(\theta_i) + \left\langle \frac{1}{N} \sum_{l=1}^N \nabla_{\theta} \mathcal{L}(x_l, \theta) \bigg|_{\theta=\theta_i}, \Delta\theta \right\rangle \quad (49)$$

Since $\mathcal{L}(\theta_i)$ is constant with respect to $\Delta\theta$, we can drop it. Let $\nabla_{\theta} \mathcal{L}_{\text{batch}} = \frac{1}{N} \sum_{l=1}^N \nabla_{\theta} \mathcal{L}(x_l, \theta_i)$. The problem simplifies to:

$$\Delta\theta^* = \arg \min_{\|\Delta\theta\| \leq \eta} \langle \nabla_{\theta} \mathcal{L}_{\text{batch}}, \Delta\theta \rangle \quad (50)$$

The solution depends on the choice of norm used for the constraint.

Deriving Adam/SignSGD via the Infinity Norm (L_{∞}) Let's constrain the step size using the L_{∞} norm: $\|\Delta\theta\|_{\infty} \leq \eta$. This is equivalent to constraining each component individually: $|\Delta\theta_j| \leq \eta$ for all j .

The objective function is $\sum_j (\nabla_{\theta} \mathcal{L}_{\text{batch}})_j \cdot (\Delta\theta_j)$. Since the objective and constraints are decoupled for each component j , we can solve for each $\Delta\theta_j$ independently:

$$(\Delta\theta_j)^* = \arg \min_{|\Delta\theta_j| \leq \eta} (\nabla_{\theta} \mathcal{L}_{\text{batch}})_j \cdot (\Delta\theta_j) \quad (51)$$

To minimize this product, we should choose $\Delta\theta_j$ to have the opposite sign of its corresponding gradient component and the largest possible magnitude, which is η .

- If $(\nabla_{\theta} \mathcal{L}_{\text{batch}})_j > 0$, we choose $\Delta\theta_j = -\eta$.

- If $(\nabla_{\theta} \mathcal{L}_{\text{batch}})_j < 0$, we choose $\Delta\theta_j = +\eta$.

This can be written concisely as:

$$\Delta\theta_j^* = -\eta \cdot \text{sign}((\nabla_{\theta} \mathcal{L}_{\text{batch}})_j) \quad (52)$$

This update rule is exactly **SignSGD**. Adam can be viewed as a smoothed version of SignSGD, where momentum is used to average the sign of the gradients over time. This shows that Adam is, in a sense, the optimal descent algorithm under an L_{∞} constraint on the step size.

Deriving Gradient Descent via the Euclidean Norm (L_2) If we instead constrain the step size using the L_2 norm, $\|\Delta\theta\|_2 \leq \eta$, the problem becomes:

$$\Delta\theta^* = \arg \min_{\|\Delta\theta\|_2 \leq \eta} \langle \nabla_{\theta} \mathcal{L}_{\text{batch}}, \Delta\theta \rangle \quad (53)$$

By the Cauchy-Schwarz inequality, the inner product $\langle u, v \rangle$ is minimized when the vectors u and v are anti-parallel (point in opposite directions). Therefore, the optimal step $\Delta\theta^*$ must be in the opposite direction of the gradient $\nabla_{\theta} \mathcal{L}_{\text{batch}}$. To satisfy the constraint, we take the largest possible step in that direction:

$$\Delta\theta^* = -\eta \frac{\nabla_{\theta} \mathcal{L}_{\text{batch}}}{\|\nabla_{\theta} \mathcal{L}_{\text{batch}}\|_2} \quad (54)$$

This is a normalized gradient descent step. The update $\theta_{i+1} = \theta_i + \Delta\theta^*$ is standard gradient descent with a learning rate that depends on η and the gradient norm.

4.3 Standardization and Normalization

Beyond the choice of optimizer, data preprocessing is crucial for successful training.

4.3.1 Why Standardize Data?

Standardizing input data (e.g., to have zero mean and unit variance) is a standard practice in machine learning for several reasons:

1. **Equal Feature Contribution:** It places all features on a similar scale, preventing features with large magnitudes from dominating the learning process and the loss function.
2. **Numerical Stability:** It avoids numerical precision issues that can arise from very large or very small values in computations. It also improves the conditioning of matrices involved in the optimization.
3. **Effective Use of Non-linearities:** Neural network activation functions are most sensitive or "interesting" in a specific range, typically around zero. Standardizing data ensures that the inputs to the first layer fall into this range, allowing the network to leverage the full expressive power of its non-linearities.

4.3.2 The Danger of Unstandardized Data: A ReLU Example

Consider a simple neuron with a ReLU activation: $h = \max(0, wx + b)$. The non-linearity (the "elbow") occurs at $x = -b/w$.

Suppose we initialize weights w and biases b from a standard normal distribution, $w, b \sim \mathcal{N}(0, 1)$. The location of the elbow, $-b/w$, follows a **Cauchy distribution**. The PDF of the standard Cauchy distribution is $p(z) = \frac{1}{\pi(1+z^2)}$.

Key properties of the Cauchy distribution:

- Most of its probability mass is concentrated around zero. For instance, $\sim 90\%$ of the mass lies between -7 and 7.
- It has heavy tails, meaning there is a non-trivial probability of sampling very large values. Its mean is undefined and its variance is infinite.

Now, suppose our input data lies in a range far from zero, e.g., $x \in [100, 200]$. The probability of an elbow $-b/w$ landing in this range is extremely small (e.g., $\approx 0.01\%$).

Consequence: For the vast majority of neurons, the pre-activation $wx + b$ will be either always positive or always negative for all data points in the training set.

- If $wx + b > 0$ for all x , the ReLU acts as a linear function: $h(x) = wx + b$.
- If $wx + b < 0$ for all x , the ReLU acts as a zero function: $h(x) = 0$.

This effectively "kills" the non-linearity and drastically reduces the network's expressive power. Mathematically, the matrix of hidden activations for all data points becomes **very low-rank**. If all neurons become affine functions $h_i(x) = \alpha_i x + \beta_i$, the resulting feature matrix has a rank of at most 2. This severely constrains the function that the next layer can learn.

4.4 Weight Initialization Strategies

Standardizing the initial input data is only the first step. We need to ensure that the inputs to *every* layer remain well-behaved, at least at initialization. This is the role of weight initialization.

Goal: Initialize weights such that the statistical properties (mean and variance) of the activations are preserved as they propagate through the network.

4.4.1 Why Not Initialize All Weights to Zero?

If all weights are initialized to zero, the input to every neuron (after the first layer) will be zero. The gradients with respect to the weights will also be zero, causing the weights to never update. The network will fail to learn.

4.4.2 Xavier (Glorot) Initialization

Consider the pre-activation z of a neuron in a fully-connected layer:

$$z = \sum_{i=1}^{d_{\text{in}}} w_i h_i + b \quad (55)$$

where h_i are the outputs from the previous layer and d_{in} is the "fan-in" (number of inputs).

Assumptions:

- The inputs h_i are standardized: $E[h_i] = 0$ and $\text{Var}(h_i) = 1$.
- The weights w_i are initialized with $E[w_i] = 0$.
- Weights w_i and inputs h_i are independent.
- The bias b is initialized to 0.

We want the output pre-activation z to also have zero mean and unit variance.

- **Mean:** $E[z] = \sum E[w_i h_i] + E[b] = \sum E[w_i] E[h_i] + 0 = 0$. The mean is preserved.
- **Variance:**

$$\begin{aligned} \text{Var}(z) &= E[z^2] - (E[z])^2 = E \left[\left(\sum_{i=1}^{d_{\text{in}}} w_i h_i \right)^2 \right] \\ &= \sum_{i=1}^{d_{\text{in}}} E[w_i^2 h_i^2] \quad (\text{cross-terms are zero due to independence and zero mean}) \\ &= \sum_{i=1}^{d_{\text{in}}} E[w_i^2] E[h_i^2] = \sum_{i=1}^{d_{\text{in}}} \text{Var}(w_i) \text{Var}(h_i) \\ &= \sum_{i=1}^{d_{\text{in}}} \text{Var}(w_i) \cdot 1 \end{aligned}$$

Assuming all weights are sampled from the same distribution, $\text{Var}(w_i) = \text{Var}(w)$. Then $\text{Var}(z) = d_{\text{in}} \cdot \text{Var}(w)$. To achieve $\text{Var}(z) = 1$, we must set the variance of the weights to:

$$\text{Var}(w) = \frac{1}{d_{\text{in}}} \quad (56)$$

This is the **Xavier initialization** rule. Weights are typically drawn from a normal or uniform distribution with mean 0 and this variance. Some versions also consider the fan-out to balance forward and backward propagation dynamics, using $\text{Var}(w) = \frac{2}{d_{\text{in}} + d_{\text{out}}}$.

4.4.3 He Initialization

Xavier initialization was derived assuming a linear activation function. However, non-linearities like ReLU alter the statistics of the activations. A ReLU activation, for zero-mean symmetric inputs, will set approximately half of its outputs to zero. This effectively halves the number of active inputs to the next layer.

The "effective fan-in" for a layer following a ReLU layer is $d_{\text{in}}/2$. To preserve unit variance, we must compensate for this. The variance calculation becomes: $\text{Var}(z) \approx (d_{\text{in}}/2) \cdot \text{Var}(w)$. To make $\text{Var}(z) = 1$, we now need:

$$\text{Var}(w) = \frac{2}{d_{\text{in}}} \tag{57}$$

This is the **He initialization** rule, which is the standard and recommended initialization for networks using ReLU or its variants (like Leaky ReLU). It accounts for the non-saturating, one-sided nature of these activation functions and has been shown to enable the training of much deeper networks.

Lecture 5: Optimizers: insights from induced matrix norms

5.1 Motivation: Why Better Optimizers?

The study of optimizers and initialization schemes is not merely about minor details; these choices have a profound impact on the performance and feasibility of training deep neural networks.

5.1.1 The Practical Importance of Optimization

The primary drivers for developing better optimizers are practical and economic:

- **Faster Training:** A better optimizer leads to faster convergence.
- **Cost Reduction:** Faster training reduces computational resource usage (e.g., GPU time), which translates directly to lower financial costs.

Similarly, principled initialization methods like Xavier and He initialization were significant breakthroughs. Before their development, a common practice was to initialize all weights from a standard normal distribution, $\mathcal{N}(0, 1)$. This often led to networks that would not train due to vanishing or exploding gradients. Figuring out the right initialization was akin to an intractable hyperparameter search over millions of parameters. These methods provided robust rules of thumb that are now standard in deep learning libraries.

5.1.2 The State of the Art and Its Challenges

Currently, the default optimizer for many deep learning applications is **AdamW**. AdamW is a variant of Adam that decouples weight decay from the adaptive learning rate mechanism, leading to better performance. The deep learning community has invested immense effort in fine-tuning the hyperparameters for AdamW on standard architectures, making it a very strong baseline.

This creates a challenge for new optimizers: a new method must outperform a highly-tuned, well-understood incumbent. Comparing a new optimizer with default hyperparameters against a fine-tuned AdamW is often not a fair comparison.

5.1.3 Lazy Training: A Case for Better Optimizers

One phenomenon observed when using optimizers like Adam is **lazy training**. This is an empirical observation where, throughout the training process, the network's parameters change very little from their random initialization, especially in the hidden layers. While the final layer might adapt, the core feature extractors remain mostly static.

This observation can be interpreted in two ways:

1. **The "Magic" Perspective (Kernel/Feature Learning):** Perhaps the random initialization is already good enough. The network is essentially using random, non-linear feature

projections, and learning only a linear model (the last layer) on top of these features is sufficient for the task. This connects to ideas like Neural Tangent Kernels (NTK).

2. **The "Inefficiency" Perspective:** It seems highly improbable that a random initialization is optimal. If the parameters are not moving, we are likely not utilizing the full expressive power of the deep network. This suggests that our current optimizers may be stuck in a regime where they cannot effectively move the parameters, pointing to a need for better optimization methods.

5.2 A Local Linear Perspective on Optimization

To develop a more principled approach to creating optimizers, we can analyze the gradient descent step from a local, linearized perspective. This viewpoint follows the work of Bernstein & Newhouse (2024), which builds upon a rich history of prior research.

Let θ be the model parameters, $\vec{x}$ be the data, and $(\vec{x}, \theta)$ be the loss function. We can approximate the loss at a nearby point $\theta_t + \Delta\theta$ using a first-order Taylor expansion around the current parameters θ_t :

$$(\vec{x}, \theta_t + \Delta\theta) \approx (\vec{x}, \theta_t) + \left. \frac{\partial}{\partial \theta} \right|_{\theta=\theta_t}^T \Delta\theta$$

This is often written more compactly as:

$$(\theta_t + \Delta\theta) \approx (\theta_t) + \langle \nabla_{\theta}(\theta_t), \Delta\theta \rangle$$

The fundamental tension in gradient-based optimization is:

- We want to take a **small step** ($\Delta\theta$) so that the local linear approximation remains valid.
- We want to take a **large step** to converge quickly.

This can be formulated as a constrained optimization problem for a single update step. We want to find the step $\Delta\theta$ that minimizes our linear approximation of the loss, while ensuring the step size is not too large.

$$\Delta\theta^* = \underset{\|\Delta\theta\| \leq \eta}{\text{argmin}} ((\theta_t) + \langle \nabla_{\theta}(\theta_t), \Delta\theta \rangle)$$

Here, η is the step size (learning rate), and the choice of norm $\|\cdot\|$ defines the geometry of the constraint region. Since (θ_t) is a constant with respect to the optimization variable $\Delta\theta$, we can simplify the problem to:

$$\Delta\theta^* = \underset{\|\Delta\theta\| \leq \eta}{\text{argmin}} \langle \nabla_{\theta}(\theta_t), \Delta\theta \rangle$$

This framework provides a "recipe" for generating optimizers: choose a norm, solve the resulting optimization problem, and the solution defines the update rule.

5.3 A Recipe for Optimizers from Norms

5.3.1 Choice 1: The ℓ_∞ Norm (Sign SGD)

Let's choose the infinity norm for our constraint: $\|\Delta\theta\|_\infty \leq \eta$. This constraint requires that the maximum absolute value of any component of $\Delta\theta$ is at most η . Geometrically, this defines a hypercube centered at the origin.

The optimization problem is:

$$\|\Delta\theta\|_\infty \leq \eta \langle \nabla_\theta(\theta_t), \Delta\theta \rangle = \|\Delta\theta\|_\infty \leq \eta \sum_j (\nabla_\theta(\theta_t))_j (\Delta\theta)_j$$

This is a **linear program**: a linear objective with linear constraints. The solution to a linear program on a bounded domain (like a hypercube) must lie at one of its vertices (corners). To minimize the inner product, we want to make each term $(\nabla_\theta(\theta_t))_j (\Delta\theta)_j$ as negative as possible. We achieve this by setting the sign of $(\Delta\theta)_j$ to be opposite to the sign of $(\nabla_\theta(\theta_t))_j$, and its magnitude to the maximum allowed value, η .

The solution is:

$$(\Delta\theta)_j = \begin{cases} +\eta & \text{if } (\nabla_\theta(\theta_t))_j < 0 \\ -\eta & \text{if } (\nabla_\theta(\theta_t))_j > 0 \end{cases}$$

This can be written compactly as:

$$\Delta\theta^* = -\eta \cdot \text{sgn}(\nabla_\theta(\theta_t))$$

This update rule is known as **Sign SGD**. It forms the basis of the momentum update in the Adam optimizer. This perspective reveals that Adam's update rule is optimal under an ℓ_∞ geometry.

5.3.2 Choice 2: The ℓ_2 Norm (Gradient Descent)

Now, let's choose the more conventional Euclidean or ℓ_2 norm: $\|\Delta\theta\|_2 \leq \eta$. This constraint defines a hypersphere of radius η .

The problem is:

$$\|\Delta\theta\|_2 \leq \eta \langle \nabla_\theta(\theta_t), \Delta\theta \rangle$$

Recall that the inner product can be expressed as $\langle \vec{x}, \vec{y} \rangle = \|\vec{x}\|_2 \|\vec{y}\|_2 \cos \phi$, where ϕ is the angle between the vectors. To minimize this value, we need $\cos \phi = -1$, which means the vectors must point in opposite directions (anti-aligned).

Therefore, the optimal direction for $\Delta\theta$ is exactly opposite to the gradient vector $\nabla_\theta(\theta_t)$. To satisfy the constraint $\|\Delta\theta\|_2 \leq \eta$, we move as far as possible in that direction, to the boundary of the hypersphere. The solution is:

$$\Delta\theta^* = -\eta \frac{\nabla_\theta(\theta_t)}{\|\nabla_\theta(\theta_t)\|_2}$$

This is **Normalized Gradient Descent**. The direction is identical to standard gradient descent,

but the step magnitude is constant (η), regardless of the gradient's magnitude.

Alternative Formulation: Regularization An alternative, unconstrained formulation can be derived using a Lagrange multiplier, which is equivalent to adding a regularization term:

$$\Delta\theta \langle \nabla_{\theta}(\theta_t), \Delta\theta \rangle + \lambda \|\Delta\theta\|_2^2$$

This objective function is convex in $\Delta\theta$. We can find the minimum by setting its gradient with respect to $\Delta\theta$ to zero:

$$\begin{aligned} \nabla_{\Delta\theta} (\langle \nabla_{\theta}(\theta_t), \Delta\theta \rangle + \lambda \|\Delta\theta\|_2^2) &= 0 \\ \nabla_{\theta}(\theta_t) + 2\lambda\Delta\theta &= 0 \\ \Delta\theta^* &= -\frac{1}{2\lambda} \nabla_{\theta}(\theta_t) \end{aligned}$$

Letting the step size be $\eta' = \frac{1}{2\lambda}$, this is exactly the standard **Gradient Descent** update. The constrained and regularized formulations generate the same family of solutions.

5.4 Incorporating Network Structure: Matrix-Based Optimization

So far, we have treated the parameters θ as a single, unstructured vector. However, neural network parameters are highly structured, consisting of weight matrices ($\mathbf{W}_l$) and bias vectors ($\vec{b}_l$) for each layer l . Scaling issues, as seen in initialization, primarily arise from the multiplicative interactions of the weight matrices.

This suggests we should reformulate our optimization problem to respect this structure. Let's focus on the weight matrices and define an update step $\Delta\mathbf{W}$ in the space of matrices.

$$\min_{\|\Delta\mathbf{W}\| \leq \eta} \langle \nabla_{\mathbf{W}}(\mathbf{W}), \Delta\mathbf{W} \rangle_F$$

The inner product for matrices is the Frobenius inner product: $\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{trace}(\mathbf{A}^T \mathbf{B})$.

5.4.1 The Shampoo Optimizer and the Spectral Norm

What is a natural norm for a matrix that acts as a linear operator scaling vectors? A good candidate is the **spectral norm** (also known as the operator 2-norm), which measures the maximum possible scaling of a unit vector.

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \max_{\|\vec{x}\|_2=1} \|\mathbf{A}\vec{x}\|_2$$

where $\sigma_{\max}(\mathbf{A})$ is the largest singular value of $\mathbf{A}$.

Let's use the spectral norm for our constraint:

$$\|\Delta\mathbf{W}\|_2 \leq \eta \langle \nabla_{\mathbf{W}}(\mathbf{W}), \Delta\mathbf{W} \rangle_F$$

Solving this problem yields an update rule related to the **Shampoo** optimizer.

5.4.2 Deriving the Shampoo Update Step

To solve the optimization problem, we want to find the matrix $\mathbf{B}$ with $\|\mathbf{B}\|_2 \leq 1$ that maximizes $\text{trace}(\mathbf{A}^T \mathbf{B})$, where $\mathbf{A} = \nabla_{\mathbf{W}}(\mathbf{W})$. The solution to the original minimization problem will then be $-\eta$ times this maximizing $\mathbf{B}$.

Let the Singular Value Decomposition (SVD) of $\mathbf{A}$ be $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. The trace term is $\text{trace}((\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T \mathbf{B}) = \text{trace}(\mathbf{V}\mathbf{\Sigma}^T \mathbf{U}^T \mathbf{B})$. Using the linearity of the trace and the outer product form of the SVD, $\mathbf{A}^T = \sum_i \sigma_i \vec{v}_i \vec{u}_i^T$:

$$\begin{aligned} \text{trace}(\mathbf{A}^T \mathbf{B}) &= \text{trace} \left(\left(\sum_i \sigma_i \vec{v}_i \vec{u}_i^T \right) \mathbf{B} \right) \\ &= \sum_i \sigma_i \text{trace}(\vec{v}_i \vec{u}_i^T \mathbf{B}) && \text{(Linearity of trace)} \\ &= \sum_i \sigma_i \text{trace}(\vec{u}_i^T \mathbf{B} \vec{v}_i) && \text{(Cyclic property of trace)} \end{aligned}$$

The term $\vec{u}_i^T \mathbf{B} \vec{v}_i$ is a scalar. By the Cauchy-Schwarz inequality and the definition of the spectral norm:

$$|\vec{u}_i^T \mathbf{B} \vec{v}_i| \leq \|\vec{u}_i\|_2 \|\mathbf{B} \vec{v}_i\|_2 \leq \|\vec{u}_i\|_2 \|\mathbf{B}\|_2 \|\vec{v}_i\|_2 \leq 1 \cdot 1 \cdot 1 = 1$$

Therefore, $\text{trace}(\mathbf{A}^T \mathbf{B}) \leq \sum_i \sigma_i$.

This upper bound is achieved when $\mathbf{B} = \mathbf{U}\mathbf{V}^T$. With this choice, $\vec{u}_i^T (\mathbf{U}\mathbf{V}^T) \vec{v}_j = \delta_{ij}$, making each term in the sum equal to σ_i . So, the matrix $\mathbf{B}$ that maximizes the inner product is $\mathbf{U}\mathbf{V}^T$.

The optimal update step is:

$$\Delta \mathbf{W}^* = -\eta (\mathbf{U}\mathbf{V}^T)$$

where $\mathbf{U}$ and $\mathbf{V}$ are the singular vectors from the SVD of the gradient matrix $\nabla_{\mathbf{W}}(\mathbf{W})$. While computing the full SVD at every step is computationally expensive, the Shampoo optimizer employs efficient approximations.

5.5 Towards a Principled Choice of Norm for Deep Learning

Our recipe for creating optimizers depends on the choice of norm. Can we find a norm that is particularly well-suited for deep neural networks by considering their architectural properties?

5.5.1 Recapping Xavier Initialization and Norms

Xavier initialization was designed to preserve the variance of signals as they propagate through the network. If the input to a neuron has variance 1, we want the output to also have variance 1. To achieve this, weights were scaled by $\frac{1}{\sqrt{d_{in}}}$, where d_{in} is the number of incoming connections (fan-in).

This principle suggests that a good "norm" or measure of magnitude should be invariant to the dimension of the vector. The standard ℓ_2 norm does not have this property; a vector in a higher-dimensional space can have a larger ℓ_2 norm even if its components are drawn from the same distribution.

5.5.2 The Root-Mean-Square (RMS) Norm

Let's define a norm that normalizes by dimension. For a vector $\vec{x} \in^d$, the **Root-Mean-Square (RMS) norm** is:

$$\|\vec{x}\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} = \frac{1}{\sqrt{d}} \|\vec{x}\|_2$$

If $\|\vec{x}\|_{\text{RMS}} = 1$, it means the "average" squared magnitude of an entry is 1. This captures the spirit of Xavier initialization.

5.5.3 The Induced RMS-to-RMS Norm

We can use this vector norm to define an induced matrix norm, which measures how a matrix scales vectors from an input space with one norm to an output space with another. For a matrix $\mathbf{A} : d_1 \rightarrow d_2$, the induced RMS-to-RMS norm is:

$$\|\mathbf{A}\|_{\text{RMS} \rightarrow \text{RMS}} = \max_{\|\vec{x}\|_{\text{RMS}}=1} \|\mathbf{A}\vec{x}\|_{\text{RMS}}$$

Let's expand the definition:

$$\begin{aligned} \|\mathbf{A}\|_{\text{RMS} \rightarrow \text{RMS}} &= \max_{\frac{1}{\sqrt{d_1}} \|\vec{x}\|_2=1} \left(\frac{1}{\sqrt{d_2}} \|\mathbf{A}\vec{x}\|_2 \right) \\ &= \max_{\|\vec{x}\|_2=\sqrt{d_1}} \left(\frac{1}{\sqrt{d_2}} \|\mathbf{A}\vec{x}\|_2 \right) \end{aligned}$$

By scaling the optimization variable, we can rewrite this with a unit-norm constraint:

$$\begin{aligned} \|\mathbf{A}\|_{\text{RMS} \rightarrow \text{RMS}} &= \frac{\sqrt{d_1}}{\sqrt{d_2}} \max_{\|\vec{z}\|_2=1} \|\mathbf{A}\vec{z}\|_2 \\ &= \sqrt{\frac{d_1}{d_2}} \|\mathbf{A}\|_2 \end{aligned}$$

This is a key result: the induced RMS-to-RMS norm is simply the **spectral norm scaled by the square root of the ratio of fan-in to fan-out**.

5.6 Conclusion and a Look Ahead

By using the induced RMS-to-RMS norm in our optimizer recipe, we can build an optimizer that naturally accounts for the specific geometry of each layer in a neural network.

Consider the constraint $\|\Delta \mathbf{W}\|_{\text{RMS} \rightarrow \text{RMS}} \leq \eta$. This is equivalent to:

$$\sqrt{\frac{d_{in}}{d_{out}}} \|\Delta \mathbf{W}\|_2 \leq \eta \implies \|\Delta \mathbf{W}\|_2 \leq \eta \sqrt{\frac{d_{out}}{d_{in}}}$$

This means that a single, global learning rate hyperparameter η results in layer-specific effective learning rates that are automatically adjusted based on the layer's dimensions (fan-in and fan-out). This can significantly simplify hyperparameter tuning, as it mitigates the problem where different layers require vastly different learning rate scales.

This is the core insight behind the **Maximal Update Parametrization (muP)**, which provides a principled way to parameterize and optimize neural networks of varying widths, a topic for the next lecture.

Lecture 6: muP: maximal update parameterization

6.1 Recap: The RMS to RMS Norm

We begin by recalling the concept of an induced matrix norm. For a matrix A , the norm induced by vector norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ is defined as:

$$\|A\|_{\alpha \rightarrow \beta} = \max_{\|\vec{x}\|_\alpha=1} \|A\vec{x}\|_\beta$$

Last time, we were motivated by Xavier initialization, which suggests that the activations in a neural network should have a consistent scale, specifically with entries of order one. This led us to the Root Mean Square (RMS) norm for a vector $\vec{x} \in \mathbb{R}^d$:

$$\|\vec{x}\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} = \frac{1}{\sqrt{d}} \|\vec{x}\|_2$$

We can use this to define an induced matrix norm, the RMS to RMS norm. Let $A \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ be a weight matrix that maps an input $\vec{x} \in \mathbb{R}^{d_{\text{in}}}$ to an output $\vec{h} \in \mathbb{R}^{d_{\text{out}}}$. The RMS to RMS norm of A is:

$$\begin{aligned} \|A\|_{\text{RMS} \rightarrow \text{RMS}} &= \max_{\|\vec{x}\|_{\text{RMS}}=1} \|A\vec{x}\|_{\text{RMS}} \\ &= \max_{\frac{1}{\sqrt{d_{\text{in}}}} \|\vec{x}\|_2=1} \frac{1}{\sqrt{d_{\text{out}}}} \|A\vec{x}\|_2 \\ &= \max_{\|\vec{x}\|_2=\sqrt{d_{\text{in}}}} \frac{1}{\sqrt{d_{\text{out}}}} \|A\vec{x}\|_2 \\ &= \frac{\sqrt{d_{\text{in}}}}{\sqrt{d_{\text{out}}}} \max_{\|\vec{z}\|_2=1} \|A\vec{z}\|_2 \quad (\text{where } \vec{z} = \vec{x}/\sqrt{d_{\text{in}}}) \\ &= \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|A\|_2 \end{aligned}$$

Here, $\|A\|_2$ is the spectral norm of A , which is its largest singular value. The key takeaway is that the RMS to RMS norm naturally incorporates the dimensions of the matrix (fan-in d_{in} and fan-out d_{out}), which aligns with the structural properties of neural network layers.

6.2 The Optimizer Recipe and Layer-Specific Rates

6.2.1 The General Recipe

Recall our general recipe for designing an optimizer. Given the gradient of the loss with respect to the weights, $\nabla_W \mathcal{L}(W)$, we find the update step ΔW by solving:

$$\arg \min_{\|\Delta W\| \leq \eta} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$$

The optimizer is defined by two choices:

1. An appropriate norm $\|\cdot\|$ for the constraint.
2. A step size hyperparameter η .

The choice of norm should reflect the geometry of the problem. However, choosing η typically requires an expensive hyperparameter sweep.

6.2.2 Using the RMS to RMS Norm

Let's apply our RMS to RMS norm to this recipe. We want to find an update ΔW such that its norm is bounded by some hyperparameter γ :

$$\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma$$

Using our derived relationship, this is equivalent to:

$$\sqrt{\frac{d_{in}}{d_{out}}} \|\Delta W\|_2 \leq \gamma \implies \|\Delta W\|_2 \leq \gamma \sqrt{\frac{d_{out}}{d_{in}}}$$

Key Observation: By choosing the RMS to RMS norm, we can use a single, global hyperparameter γ for the entire network. However, the constraint on the spectral norm of the update for each layer automatically scales with the layer's dimensions (d_{in} and d_{out}). This provides a principled way to achieve layer-specific update magnitudes without needing to tune separate hyperparameters for each layer.

This exact idea, when plugged into the optimizer recipe, leads to a simplified version of the Shampoo optimizer and is the foundation for the **Muon optimizer**. Muon has shown tremendous practical impact, significantly accelerating training time for models like NanoGPT and enabling the training of trillion-parameter models.

6.3 Deriving an Optimal Step-Size Scaling for Adam/SignSGD

Instead of building a new optimizer, can we use this insight to improve existing, widely-used optimizers like Adam or SignSGD? We will focus on SignSGD for simplicity, whose update rule is:

$$W_{t+1} = W_t - \eta \cdot \text{sign}(\nabla_W \mathcal{L}(W))$$

Our goal is to find a principled way to choose the learning rate η by factoring in the layer geometry. We will do this by requiring that the update step $\Delta W = -\eta \cdot \text{sign}(\nabla_W \mathcal{L}(W))$ has a bounded RMS to RMS norm:

$$\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma$$

6.3.1 Analysis with Mini-Batch Size 1

Let's consider an update based on a single data point (mini-batch size 1).

Gradient Rank For a single data point and common loss functions (e.g., squared error, cross-entropy), the gradient with respect to the weights, $\nabla_W \mathcal{L}$, is a rank-one matrix. This is because the update must lie in the span of the input activations. Using the chain rule, for a layer $\vec{h} = W\vec{x}$, we have $\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \vec{h}} \otimes \vec{x}$, which is an outer product and thus rank-one. We can write it as:

$$\nabla_W \mathcal{L} = \sigma \vec{u} \vec{v}^T$$

For a rank-one matrix, the spectral norm and Frobenius norm are equal: $\|\nabla_W \mathcal{L}\|_2 = \|\nabla_W \mathcal{L}\|_F = \sigma$.

Sign Matrix Rank Let $S = \text{sign}(\nabla_W \mathcal{L})$. Since the sign function is applied element-wise and $\text{sign}(ab) = \text{sign}(a)\text{sign}(b)$, we have:

$$S = \text{sign}(\sigma \vec{u} \vec{v}^T) = \text{sign}(\vec{u}) \text{sign}(\vec{v}^T)$$

This shows that S is also an outer product of two vectors, and therefore is also a rank-one matrix.

Deriving the Scaling for η Now we can analyze our constraint $\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \gamma$.

$$\| -\eta S \|_{\text{RMS} \rightarrow \text{RMS}} = \eta \|S\|_{\text{RMS} \rightarrow \text{RMS}} = \eta \sqrt{\frac{d_{in}}{d_{out}}} \|S\|_2 \leq \gamma$$

We need to compute the spectral norm of S . Since S is rank-one, its spectral norm is equal to its Frobenius norm. The matrix $S \in \mathbb{R}^{d_{out} \times d_{in}}$ has entries that are all ± 1 .

$$\|S\|_F^2 = \sum_{i=1}^{d_{out}} \sum_{j=1}^{d_{in}} S_{ij}^2 = \sum_{i=1}^{d_{out}} \sum_{j=1}^{d_{in}} (\pm 1)^2 = d_{out} \cdot d_{in}$$

Therefore, $\|S\|_2 = \|S\|_F = \sqrt{d_{in} d_{out}}$. Substituting this back into our inequality:

$$\begin{aligned} \eta \sqrt{\frac{d_{in}}{d_{out}}} \left(\sqrt{d_{in} d_{out}} \right) &\leq \gamma \\ \eta \cdot d_{in} &\leq \gamma \\ \eta &\leq \frac{1}{d_{in}} \gamma \end{aligned}$$

This result provides a principled scaling law: **the learning rate η for a given layer should be scaled inversely with its fan-in dimension, d_{in} .**

Note on Larger Batch Sizes When the batch size is greater than 1, the gradient is no longer guaranteed to be rank-one. However, the inequality $\|A\|_2 \leq \|A\|_F$ always holds. If we use the

Frobenius norm as an upper bound for the spectral norm in our derivation, the analysis follows, but the resulting bound on η will be more conservative (i.e., suggest a smaller learning rate).

6.4 Maximal Update Parameterization (μ P)

The scaling law we just derived is the essence of Maximal Update Parameterization (μ P), a framework for understanding how to scale hyperparameters as model dimensions change.

6.4.1 The Hyperparameter Transfer Problem

Training large neural networks is extremely expensive. A key challenge is hyperparameter tuning. Empirically, the optimal learning rate for a model changes as its width (e.g., d_{model}) changes. Under standard parameterizations (like Xavier initialization alone), the optimal learning rate can shift by orders of magnitude, making it impossible to tune on a small, cheap "proxy" model and transfer the settings to a full-scale model.

μ P provides a way to parameterize the model such that the optimal hyperparameters (especially learning rate) remain stable across different model widths. This enables "one-shot transfer": find the best hyperparameters on a small model and use them directly on the large model.

6.4.2 Analogy: The Central Limit Theorem

To understand the importance of correct scaling, consider the sum of n i.i.d. random variables X_i with zero mean and unit variance.

- $\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{n \rightarrow \infty} 0$ (Wrong scaling)
- $\sum_{i=1}^n X_i \xrightarrow{n \rightarrow \infty} \infty$ (Wrong scaling)
- $\frac{1}{\sqrt{n}} \sum_{i=1}^n X_i \xrightarrow{d} \mathcal{N}(0, 1)$ (Correct scaling)

Only with the correct scaling factor, $1/\sqrt{n}$, does the quantity converge to a non-trivial limit. If we define a function $F_n(c) = \mathbb{E}[f(c \sum X_i)]$ and reparameterize it with $c = \alpha/\sqrt{n}$ to get $G_n(\alpha) = F_n(\alpha/\sqrt{n})$, then as $n \rightarrow \infty$, $G_n(\alpha)$ converges to a function $G(\alpha)$ that is independent of n . Therefore, the optimal α^* that minimizes $G_n(\alpha)$ will be stable for large n . This allows us to find α^* on a smaller n and transfer it to a larger N .

μ P aims to find the "right scaling" for neural network hyperparameters with respect to network width, just as $1/\sqrt{n}$ is the right scaling for the sum of random variables.

6.5 A Principled View: Conditions for Stable Feature Learning

A simplified perspective on μ P comes from identifying two desired properties (desiderata) for the network's behavior as its width changes. We want both the activations and their updates to maintain a stable scale.

Let $\vec{h}_l$ be the hidden activations at layer l . The two desiderata are:

1. **Stable Activation Scale:** The RMS norm of the activations should be of constant order with respect to network width: $\|\vec{h}_l\|_{\text{RMS}} = \Theta(1)$.
2. **Stable Update Scale:** The RMS norm of the change in activations per optimizer step should also be of constant order: $\|\Delta\vec{h}_l\|_{\text{RMS}} = \Theta(1)$.

6.5.1 Achieving the Desiderata

Ignoring bias and non-linearities, a layer computes $\vec{h}_l = W_l \vec{h}_{l-1}$.

1. To satisfy the first desideratum, we need to preserve the RMS norm across layers. An inductive argument shows that if $\|\vec{h}_{l-1}\|_{\text{RMS}} = \Theta(1)$, then to get $\|\vec{h}_l\|_{\text{RMS}} = \Theta(1)$, we must have $\|W_l\|_{\text{RMS} \rightarrow \text{RMS}} = \Theta(1)$. This is precisely the condition that motivates Xavier initialization.
2. To satisfy the second desideratum, consider the update to the activations: $\Delta\vec{h}_l = (\Delta W_l) \vec{h}_{l-1} + W_l(\Delta\vec{h}_{l-1})$. For $\|\Delta\vec{h}_l\|_{\text{RMS}}$ to remain $\Theta(1)$ throughout the network, the updates from the current layer, $(\Delta W_l) \vec{h}_{l-1}$, must have the same scale as the propagated updates, $W_l(\Delta\vec{h}_{l-1})$. This requires the norm of the weight update, $\|\Delta W_l\|$, to have the same scaling as the weight norm, $\|W_l\|$, with respect to the layer dimensions.

This leads to two fundamental conditions for stable feature learning at scale:

- **Initialization Condition:** $\|W_l\|_{\text{RMS} \rightarrow \text{RMS}} = \Theta(1)$
- **Optimization Condition:** $\|\Delta W_l\|_{\text{RMS} \rightarrow \text{RMS}} = \Theta(1)$

The second condition provides a fundamental justification for our earlier approach: constraining the optimizer update step in the RMS to RMS norm is a principled way to ensure stable training dynamics as network width grows.

Lecture 7: Optimizers: MuON

7.1 Recap: Maximal Update Parametrization (MUP)

The primary goal of Maximal Update Parametrization (MUP) is to address the challenge of hyperparameter tuning for very large neural networks. Training these models is computationally expensive, making traditional hyperparameter sweeps infeasible. MUP provides a principled way to find optimal hyperparameters on a smaller, cheaper "proxy" network and then transfer them directly to a much larger target network. This lecture focuses on scaling with respect to network **width**.

7.1.1 A Spectral Perspective on Feature Learning

To build intuition, we adopt a simplified linear algebraic perspective from the paper "A Spectral Condition for Feature Learning" by Bernstein, Simon, and Yang. This perspective leads to the same scaling laws as the original MUP paper, which used more complex tensor algebra.

Consider a single hidden layer L in a network:

$$H_L = \phi(W_L H_{L-1} + b_L)$$

where H_L is the output activation vector, $W_L \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix, H_{L-1} is the input activation vector, b_L is the bias, and ϕ is a non-linear activation function.

For effective "feature learning" (i.e., the network's parameters move meaningfully from their initialization), we desire two properties to hold as the network width scales:

1. **Stable Activation Magnitudes:** The activations at each layer should have a consistent magnitude, independent of the network width. We formalize this by requiring the Root Mean Square (RMS) norm of the hidden layers to be of order one.

$$\|H_L\|_{RMS} = \sqrt{\frac{1}{d_{out}} \sum_{i=1}^{d_{out}} (H_L)_i^2} \approx O(1)$$

Since $\|H_L\|_{RMS} = \frac{1}{\sqrt{d_{out}}} \|H_L\|_2$, this is equivalent to requiring the Euclidean norm to be $\|H_L\|_2 \approx O(\sqrt{d_{out}})$.

2. **Meaningful Parameter Updates:** The updates to the parameters during training should not become negligible as the network width increases. If they do, the network remains stuck near its initialization. We formalize this by requiring the RMS norm of the change in activations, ΔH_L , to also be of order one.

$$\|\Delta H_L\|_{RMS} \approx O(1) \implies \|\Delta H_L\|_2 \approx O(\sqrt{d_{out}})$$

7.1.2 Deriving Scaling Conditions

To see what conditions on the weights W_L achieve these properties, we simplify the model by ignoring the bias and non-linearity:

$$H_L = W_L H_{L-1}$$

We proceed inductively, assuming the desired properties hold for layer $L - 1$. The paper argues that during training, the hidden state vectors H_{L-1} tend to align with the top singular vectors of the weight matrix W_L . This allows us to approximate the effect of the matrix multiplication on the norm:

$$\|H_L\|_2 \approx \|W_L\|_{spec} \cdot \|H_{L-1}\|_2$$

where $\|W_L\|_{spec} = \sigma_{max}(W_L)$ is the spectral norm.

Using our desired properties for the Euclidean norms, we substitute them into the equation:

$$O(\sqrt{d_{out}}) \approx \|W_L\|_{spec} \cdot O(\sqrt{d_{in}})$$

This implies a condition on the spectral norm of the weight matrix:

$$\|W_L\|_{spec} \approx O\left(\sqrt{\frac{d_{out}}{d_{in}}}\right)$$

This is equivalent to the condition from the previous lecture that the RMS-to-RMS induced norm of W_L should be $O(1)$.

Next, we analyze the update ΔH_L . The update rule for H_L is:

$$\Delta H_L = \Delta W_L \cdot H_{L-1} + W_L \cdot \Delta H_{L-1}$$

Taking the norm and using the triangle inequality (and again, the top singular vector alignment assumption):

$$\|\Delta H_L\|_2 \approx \|\Delta W_L \cdot H_{L-1} + W_L \cdot \Delta H_{L-1}\|_2 \leq \|\Delta W_L\|_{spec} \|H_{L-1}\|_2 + \|W_L\|_{spec} \|\Delta H_{L-1}\|_2$$

We want $\|\Delta H_L\|_2 \approx O(\sqrt{d_{out}})$. Inductively, we assume $\|H_{L-1}\|_2 \approx O(\sqrt{d_{in}})$ and $\|\Delta H_{L-1}\|_2 \approx O(\sqrt{d_{in}})$. We already derived that $\|W_L\|_{spec} \approx O(\sqrt{d_{out}/d_{in}})$. For the equation to balance, the remaining term must also satisfy:

$$\|\Delta W_L\|_{spec} \approx O\left(\sqrt{\frac{d_{out}}{d_{in}}}\right)$$

This implies that both the weight matrix W_L and its update ΔW_L must have an RMS-to-RMS induced norm of $O(1)$. These conditions are the foundation for deriving the MUP scaling laws for hyperparameters like the learning rate.

7.2 Muon: Momentum Orthogonalized by Newton-Schulz

7.2.1 Motivation: State-of-the-Art Performance

Muon is a recent, powerful optimizer that has demonstrated significant performance gains. Its introduction was a key factor in a massive improvement in the "NanoGPT speedrun," a competition to train a small language model as quickly as possible. The ideas have also been successfully applied to much larger, trillion-parameter models, indicating their scalability and relevance to frontier models.

7.2.2 Revisiting the Optimizer Recipe and Shampoo

Recall the general recipe for a first-order optimizer. At each step, we want to find an update ΔW that maximally decreases the loss, constrained by some norm:

$$\Delta W^* = \arg \min_{\|\Delta W\| \leq \eta} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$$

The solution is to align ΔW with the negative gradient: $\Delta W \propto -\nabla_W \mathcal{L}(W)$. The choice of norm $\|\cdot\|$ determines the optimizer's behavior.

The **Shampoo** optimizer can be understood by choosing the **spectral norm**, $\|\cdot\|_2$. If the gradient is $\nabla_W \mathcal{L}(W) = G_t = U\Sigma V^T$, the optimal update is a **semi-orthogonal update**:

$$\Delta W^* = \eta \cdot U_r V_r^T$$

where U_r, V_r correspond to the non-zero singular values. This update removes the influence of the singular values Σ , effectively taking a uniform step in all singular directions. This is the update for Shampoo *without accumulation*.

The actual Shampoo algorithm looks more complex, using left (L_t) and right (R_t) preconditioners:

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$$

where L_t and R_t are typically accumulated statistics of the gradients. Without accumulation, we have $L_t = G_t G_t^T$ and $R_t = G_t^T G_t$. Let's show this is equivalent to the semi-orthogonal update.

$$\begin{aligned} G_t G_t^T &= (U\Sigma V^T)(U\Sigma V^T)^T = U\Sigma V^T V \Sigma^T U^T = U\Sigma \Sigma^T U^T \\ G_t^T G_t &= (U\Sigma V^T)^T (U\Sigma V^T) = V \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V^T \end{aligned}$$

The matrix square roots act only on the diagonal singular value matrices:

$$\begin{aligned} (G_t G_t^T)^{-1/4} &= (U(\Sigma \Sigma^T)^{1/2} U^T)^{-1/2} = U(\Sigma \Sigma^T)^{-1/4} U^T = U \Sigma^{-1/2} U^T \\ (G_t^T G_t)^{-1/4} &= V(\Sigma^T \Sigma)^{-1/4} V^T = V \Sigma^{-1/2} V^T \end{aligned}$$

(Assuming a square, invertible Σ for simplicity). Combining these:

$$(G_t G_t^T)^{-1/4} G_t (G_t^T G_t)^{-1/4} = (U \Sigma^{-1/2} U^T) (U \Sigma V^T) (V \Sigma^{-1/2} V^T) = U \Sigma^{-1/2} \Sigma \Sigma^{-1/2} V^T = U V^T$$

The fourth roots are used because they can be computed more efficiently than a full SVD using iterative Newton-style methods.

7.2.3 Key Idea 1: RMS-to-RMS Norm Optimization

Muon's first key idea is to replace the spectral norm in the optimizer recipe with the **RMS-to-RMS induced norm**, inspired by MUP. The RMS-to-RMS norm is just a scaled version of the spectral norm:

$$\|\Delta W\|_{RMS \rightarrow RMS} = \sqrt{\frac{d_{in}}{d_{out}}} \|\Delta W\|_2$$

The optimization problem becomes:

$$\arg \min_{\|\Delta W\|_{RMS \rightarrow RMS} \leq \eta} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle \quad \equiv \quad \arg \min_{\|\Delta W\|_2 \leq \eta \sqrt{\frac{d_{out}}{d_{in}}}} \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$$

The solution is a scaled semi-orthogonal update:

$$\Delta W^* = \eta \sqrt{\frac{d_{out}}{d_{in}}} U_r V_r^T$$

This scaling factor, derived from MUP principles, is the first major component of Muon.

7.2.4 Key Idea 2: Efficient Semi-Orthogonalization via Newton-Schulz

Muon's second key idea is a computationally efficient method to approximate the semi-orthogonal matrix UV^T without performing an SVD. This is achieved through **Newton-Schulz iterations**.

The goal is to find a function f such that for a matrix $X = U \Sigma V^T$, we have $f(X) \approx UV^T$. The core insight is that **odd polynomials commute with the SVD**. For an odd polynomial $p(x) = a_1 x + a_3 x^3 + \dots$, we have:

$$p(U \Sigma V^T) = U p(\Sigma) V^T$$

This means the polynomial acts directly on the singular values, while leaving the singular vectors U and V unchanged. The problem reduces to finding a scalar polynomial $p(x)$ that, when iterated, drives its input towards 1 (for positive inputs).

A polynomial with this property is:

$$p(x) = \frac{3}{2}x - \frac{1}{2}x^3$$

By repeatedly applying this function, $x_{k+1} = p(x_k)$, any initial value $x_0 \in (0, \sqrt{3})$ will converge to 1. Since singular values are always non-negative, we only care about the positive domain.

However, if a singular value is greater than $\sqrt{3}$, the iteration will diverge. To prevent this, Muon employs a simple trick: normalize the gradient matrix by its Frobenius norm before applying the iterations.

$$G_{norm} = \frac{G_t}{\|G_t\|_F}$$

Since $\|G_t\|_F = \sqrt{\sum \sigma_i^2} \geq \sigma_{max} = \|G_t\|_2$, this normalization ensures that all singular values of G_{norm} are less than or equal to 1, placing them safely within the convergence basin of the polynomial iteration.

7.2.5 The Muon Algorithm and Implementation Details

Muon stands for **M**omentum **O**rthogonalized by **N**ewton-Schulz. The algorithm proceeds as follows:

1. **Momentum Step:** Accumulate gradients using a standard momentum buffer.

$$B_t = \mu B_{t-1} + G_t$$

2. **Orthogonalization Step:** Compute the approximate semi-orthogonal matrix of the momentum buffer using Newton-Schulz iterations. Let $p^{(k)}(X)$ be k applications of the polynomial.

$$O_t = p^{(k)}\left(\frac{B_t}{\|B_t\|_F}\right)$$

A typical value for k is 5.

3. **Update Step:** Update the weights using the orthogonalized direction, scaled by the learning rate and the MUP factor.

$$W_{t+1} = W_t - \eta \sqrt{\frac{d_{out}}{d_{in}}} O_t$$

Choosing the polynomial $p(x)$: The choice of polynomial is a hyperparameter. There is a trade-off:

- Higher-order polynomials may converge in fewer iterations (k can be smaller).
- However, each iteration is more computationally expensive (requires more matrix multiplications).

Interestingly, the polynomial used in the record-setting NanoGPT speedrun was a quintic:

$$p(x) = 3.444x - 4.7750x^3 + 2.0315x^5$$

This polynomial does not actually converge to 1 (since $p(1) \neq 1$). It provides only an *approximate* orthogonalization. It was chosen empirically because it has a very steep slope at $x = 0$, which aggressively scales up small singular values. This was found to be crucial for accelerating training, suggesting that perfect orthogonalization is not necessary and that sacrificing convergence for faster initial progress can be beneficial.

7.3 Conclusion and Future Directions

Muon represents the state-of-the-art in optimizers, building on ideas from Shampoo (semi-orthogonal updates) and MUP (principled hyperparameter scaling). The use of Newton-Schulz iterations provides a computationally feasible way to implement these ideas.

This is a very active area of research. Key practical observations and research questions include:

- **Batch Size:** Muon performs better with larger batch sizes, which lead to more stable gradient estimates.
- **Variants:** Many other variants exist (e.g., Scion, Prawn) that explore different polynomials or implementation details.
- **Fair Comparison:** Comparing optimizers is notoriously difficult. A fair comparison requires extensive, independent hyperparameter tuning for each optimizer on a given task, which is often not done, leading to conflicting reports in the literature.

8.1 Introduction to Architectures

Lecture 8: Conv-nets; basics

This lecture transitions from optimization to the study of neural network architectures. The sequence of architectures covered in this course will be:

1. Multi-Layer Perceptrons (MLPs)
2. Convolutional Neural Networks (CNNs)
3. Graph Neural Networks (GNNs)
4. Recurrent Neural Networks (RNNs) and State-Space Models (SSMs)
5. Transformers

8.1.1 MLPs vs. CNNs: A High-Level View

A key way to understand a new architecture is to contrast it with one we already know.

- **Multi-Layer Perceptron (MLP):** In an MLP layer, every input neuron is connected to every output neuron. This is represented by a dense, unstructured weight matrix where every element is a unique, learnable parameter.
- **Convolutional Neural Network (CNN):** A CNN layer introduces a strong structural prior into the weight matrix. It can be seen as a specialized version of an MLP layer with two defining properties:
 1. **Sparsity (Local Connectivity):** Each output neuron is connected to only a small, local region of the input neurons. This corresponds to a weight matrix that is sparse, with many zero entries representing the absence of connections.
 2. **Parameter (Weight) Sharing:** The same set of weights (known as a filter or kernel) is used for every local connection across the entire input. This means many non-zero entries in the equivalent weight matrix are identical. This drastically reduces the number of unique parameters compared to an MLP.

8.2 Motivation and Origins of CNNs

CNNs were largely inspired by and developed for **computer vision (CV)** tasks.

8.2.1 Input Data: Images

The primary data type for CV is the image. An image is represented as a 3D tensor with dimensions:

- **Height (H)**
- **Width (W)**
- **Channels (C)**

For a standard color image, there are typically 3 channels (Red, Green, Blue), so the shape is $H \times W \times 3$. CNNs are designed to operate on and preserve this spatial tensor structure for as long as possible.

8.2.2 Typical Computer Vision Tasks

- **Image Classification:** Assigning a single label to an entire image. For example, determining if an image from the CIFAR-10 dataset is a ‘cat’, ‘dog’, ‘car’, etc.
- **Semantic Segmentation:** A more complex task where every pixel in the input image is assigned a class label (e.g., ‘couch’, ‘table’, ‘person’). This requires producing an output that preserves spatial information.

Note: While this is not a computer vision course, understanding the basics of CV and Natural Language Processing (NLP) is crucial because many foundational deep learning advancements originate from these fields.

8.3 Core Principles of CNNs

The design of CNNs is driven by a set of core ideas. These can be categorized into principles related to **expressivity** (what the model can represent) and principles related to **practicality** (making the model trainable).

8.3.1 Expressivity: Building in Prior Knowledge

These principles are priors about the structure of visual data that are encoded into the architecture.

Locality

- **Premise:** The spatial arrangement of pixels is fundamentally important. Information is local; nearby pixels are strongly correlated, forming meaningful patterns like edges, textures, and corners. Shuffling the pixels of an image destroys its meaning.
- **Architectural Consequence:** The use of a **convolutional structure with small filters** (e.g., 3x3, 5x5). Each filter acts as a local pattern detector.

Invariances, Equivariances, and Symmetries

- **Premise:** The identity of an object or feature is often invariant (or equivariant) to certain transformations. For example, a cat is still a cat if it is shifted 10 pixels to the right in an image. This is known as **translation invariance/equivariance**.
- **Architectural Consequences:**

1. **Weight Sharing:** This is the direct implementation of translation equivariance. The same filter (a set of weights) is applied at every spatial position in the input. This allows a feature detector (e.g., for a horizontal edge) learned in one part of the image to be immediately reusable across the entire image, without having to re-learn it for each location.
2. **Data Augmentation:** This is a *training technique*, not an architectural feature. It is used to teach the model other invariances that are not naturally built in, such as invariance to small rotations, scaling, or lighting changes. By showing the model augmented versions of the training data, we force it to learn representations that are robust to these transformations.

Hierarchical Structure

- **Premise:** The visual world is compositional; simple parts combine to form more complex wholes. For instance, edges combine to form corners and contours, which form parts of objects (like an eye), which in turn form larger objects (like a face).
- **Architectural Consequences:**
 1. **Depth:** Stacking multiple convolutional layers allows the network to learn a hierarchy of features. Early layers learn simple features like edges and colors. Deeper layers combine these simple features to detect more complex and abstract concepts.
 2. **Multi-resolution and Filter Banks:** The idea that different features are relevant at different spatial scales. This often manifests in architectures that process the image at multiple resolutions (e.g., through downsampling) and use a varying number of filters (a "filter bank") at different layers, reflecting the increasing complexity and variety of features at higher levels of abstraction.

8.3.2 Practicalities: Getting the Network to Work

These are components and techniques developed primarily to enable stable and effective training of deep networks.

- **Normalization Layers:** (e.g., Batch Normalization, Layer Normalization) These layers do not add expressive power but are crucial for stabilizing training. They control the statistics (e.g., mean and variance) of activations between layers, preventing issues like vanishing or exploding gradients and ensuring that quantities like the RMS norm of layer inputs remain controlled.
- **Dropout:** A regularization technique that randomly sets a fraction of neuron activations to zero during training to prevent overfitting.
- **Residual/Skip Connections:** A powerful technique that adds the input of a layer (or block of layers) to its output. This makes it much easier to train very deep networks by improving gradient flow.

8.4 The Convolution Operation in Deep Learning

8.4.1 Convolution vs. Cross-Correlation

In signal processing, the 1D discrete convolution of an input signal x and a filter h is defined as:

$$(x * h)[t] = \sum_{k=-\infty}^{\infty} x[k]h[t - k]$$

The key detail is the $t - k$ term, which implies that the filter h is "flipped" before being slid across the signal.

In deep learning, the operation used is technically **cross-correlation**, where the filter is not flipped:

$$y[t] = \sum_k x[t + k]h[k]$$

Since the filter weights $h[k]$ are learned parameters, the network can simply learn the "flipped" version of the filter if that is optimal. Therefore, by convention, this operation is referred to as **convolution** in the deep learning literature.

8.4.2 Mechanics of a Convolutional Layer

A convolutional layer processes an input tensor (called a feature map) to produce an output feature map.

Handling Multiple Channels Let the input feature map have shape $H_{in} \times W_{in} \times C_{in}$.

- **Filters:** A single filter has dimensions $K_h \times K_w \times C_{in}$. Note that the depth of the filter must match the number of input channels.
- **Operation for a single output channel:**
 1. The $K_h \times K_w \times C_{in}$ filter is slid over the input feature map.
 2. At each spatial location (i, j) , a dot product is computed between the filter weights and the corresponding $K_h \times K_w \times C_{in}$ patch of the input. This involves performing a 2D convolution for each channel and then summing the results.
 3. A single scalar bias term is added to this sum.
 4. The result is the value at position (i, j) of the single output channel.
- **Multiple Output Channels:** To produce an output feature map with C_{out} channels, we use C_{out} independent filters. Each filter has its own set of weights and its own scalar bias. Each filter produces one channel of the output feature map.

Key Principle: Weights are shared across spatial locations (H, W) but are **not** shared across channels (C) .

Special Case: 1x1 Convolution A 1x1 convolution uses a filter of size $1 \times 1 \times C_{in}$.

- It operates on a single pixel location at a time, considering all C_{in} channels for that pixel.
- For a single pixel, the input is a vector $\vec{x}_{\text{pixel}} \in \mathbb{R}^{C_{in}}$.
- To produce an output with C_{out} channels, we use C_{out} different 1×1 filters.
- This entire operation is equivalent to a **dense (fully-connected) linear layer** being applied independently at every pixel location. The weight matrix is $W \in \mathbb{R}^{C_{out} \times C_{in}}$ and the bias is $\vec{b} \in \mathbb{R}^{C_{out}}$.
- The output at a pixel, followed by a non-linearity, is: $\vec{y}_{\text{pixel}} = \text{ReLU}(W\vec{x}_{\text{pixel}} + \vec{b})$.

This perspective is useful for connecting concepts from MLP analysis (like μP) to convolutional networks.

8.5 Architectural Details and Hyperparameters

8.5.1 Handling Boundaries: Padding

When a filter is applied at the edge of an input, it extends beyond the boundary. This leads to two issues:

1. The output feature map is smaller than the input.
2. The pixels at the borders are used less frequently in computations than the interior pixels.

The solution is **padding**: adding extra pixels around the border of the input.

- **Zero Padding:** The most common method, where the padding pixels have a value of zero.
- **Valid Padding:** No padding is used. The output size shrinks. If the input is $W \times W$ and the kernel is $K \times K$, the output is $(W - K + 1) \times (W - K + 1)$.
- **Same Padding:** Padding is added specifically so that the output feature map has the same spatial dimensions (H, W) as the input (assuming a stride of 1).
- **Other types:** Mirror/reflection padding or wrap-around (circular) padding can avoid creating artificial sharp edges at the boundary but are less common in practice.

8.5.2 Receptive Field

The **receptive field** of a neuron in a given layer is the region in the original input image that influences the value of that neuron.

- In a single convolutional layer with a $K \times K$ filter, the receptive field is $K \times K$.
- As layers are stacked, the receptive field grows. For a stack of L layers of 3×3 convolutions (in 1D), the receptive field size is $2L + 1$. This is a **linear growth** with respect to depth.

8.5.3 Downsampling via Stride

To increase the receptive field growth rate and reduce computational cost, we can downsample the feature maps. One way to do this is with a **stride**.

- **Definition:** The stride is the step size the filter takes as it moves across the input. A stride of 1 moves the filter one pixel at a time. A stride of 2 moves it two pixels at a time, skipping every other location.
- **Effect:** A stride greater than 1 results in a smaller output feature map, effectively combining convolution and downsampling into a single efficient operation. This leads to a faster, more multiplicative growth of the receptive field.

Lecture 9: Pooling/Downsampling, Data Augmentation, Normalization Layers

9.1 Recap and Details of Convolutional Layers

9.1.1 The Convolution Operation

Recall the basic picture of a convolution. The topology of the input matters; local neighborhoods are fundamental.

- **1D Case:** A sliding window (filter or kernel) moves across a 1D input. For each position, the values in the window are multiplied by corresponding weights in the filter, summed up, and a bias is added to produce a single output value.
- **2D Case:** This extends the 1D case. The neighborhood is now a 2D patch (e.g., 3×3). The filter is a 2D matrix of weights. The operation is the same: element-wise multiplication and summation over the patch, plus a bias.
- **Multi-Channel Input:** If the input has multiple channels (e.g., an RGB image has 3 channels), the filter must also have a depth corresponding to the number of input channels. For a 3×3 filter on an RGB image, the filter is actually a $3 \times 3 \times 3$ tensor of weights. Each channel of the input is convolved with its corresponding channel in the filter, and the results are all summed together with a single bias to produce a single value for one output channel.

Notation Note: In the context of CNNs, the filters are often called **kernels**. This is a different usage of the word "kernel" than in kernel methods like Support Vector Machines (SVMs).

9.1.2 Understanding The Number of Parameters

Let's break down the number of learnable parameters (weights and biases) in different convolutional layer configurations. Let C_{in} be the number of input channels and C_{out} be the number of output channels.

1. 1x1 convolution with 1 input channel and 1 output channel:

- For each pixel, the operation is simply scaling the input value by a constant and adding a bias.
- **Weights:** 1
- **Biases:** 1

2. 3x3 convolution with 1 input channel and 1 output channel:

- For each pixel, the values in its 3×3 neighborhood are scaled by individual, distinct weights. These are summed, and a single bias is added.

- The filter consists of 9 weights (one for the center, N, S, E, W, NE, NW, SE, SW neighbors).
- **Weights:** $3 \times 3 = 9$
- **Biases:** 1

3. **3x3 convolution with C_{in} input channels and 1 output channel:**

- The process from the previous case is repeated for each of the C_{in} input channels. Each input channel has its own distinct 3×3 filter.
- The results from all C_{in} convolutions are summed up, and a single bias is added to produce the final single output value.
- **Why only one bias?** If we had C_{in} separate biases, one for each channel's convolution, they would all be summed together into a single scalar anyway. In terms of expressive power, a single bias is equivalent. From a backpropagation perspective, the gradient would be identical for all C_{in} hypothetical biases.
- **Weights:** $9 \times C_{in}$
- **Biases:** 1

4. **$k \times k$ convolution with C_{in} input channels and 1 output channel:**

- Generalizing the above case.
- **Weights:** $k^2 \times C_{in}$
- **Biases:** 1

5. **1x1 convolution with C_{in} input channels and C_{out} output channels:**

- At each pixel, the input is a vector of size C_{in} (the values across all channels). The output is a vector of size C_{out} .
- The operation that transforms a vector of size C_{in} to a vector of size C_{out} is a matrix-vector multiplication plus a bias vector. Let $\vec{x}$ be the input vector and $\vec{y}$ be the output vector: $\vec{y} = \mathbf{W}\vec{x} + \vec{b}$, where $\mathbf{W}$ is a $C_{out} \times C_{in}$ matrix and $\vec{b}$ is a C_{out} -dimensional vector.
- **Weights:** $C_{in} \times C_{out}$
- **Biases:** C_{out}

6. **$k \times k$ convolution with C_{in} input channels and C_{out} output channels:**

- This is the most general case. It combines the logic from all previous examples. For each of the C_{out} output channels, we perform a complete $k \times k$ convolution over all C_{in} input channels.
- Each output channel is generated by its own unique set of filters. There are C_{out} sets of filters, and each set contains C_{in} filters of size $k \times k$.
- **Weights:** $k^2 \times C_{in} \times C_{out}$. These can be thought of as a 4D tensor of shape (C_{out}, C_{in}, k, k) .
- **Biases:** C_{out} (one for each output channel).

9.2 Backpropagation in CNNs

A key feature of CNNs is **weight sharing**: the same filter (parameter w) is used at many different locations in the input. This means the parameter w appears in multiple places in the computational graph. How do we compute the gradient $\frac{\partial \mathcal{L}}{\partial w}$?

9.2.1 The Conceptual Approach

1. **Pretend weights are distinct:** Imagine that each application of the filter uses a unique, independent weight $(w_1, w_2, \dots, w_N)$, which all just happen to have the same initial value, $w_i = w$.
2. **Compute individual gradients:** During backpropagation, you would compute the gradient of the loss with respect to each of these hypothetical independent weights: $\frac{\partial \mathcal{L}}{\partial w_1}, \frac{\partial \mathcal{L}}{\partial w_2}, \dots$.
3. **Sum the gradients:** To get the final gradient for the single shared weight w , you sum up all the individual gradients. This is a direct consequence of the multivariate chain rule.

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_i \frac{\partial \mathcal{L}}{\partial w_i}$$

This summation is analogous to the principle of superposition in physics. Each application of the weight contributes linearly to the total change in the loss, so their effects on the gradient add up.

9.2.2 Practical Considerations: Sum vs. Average

The gradient is a sum, not an average. The reason is that a small change in w affects the output at *every* location it is applied. The total effect on the loss is the sum of these individual effects.

However, this summation can cause the magnitude of the gradient to grow significantly, especially for large images where a filter is applied many times.

- If the individual gradients are uncorrelated, the standard deviation of the sum grows like $\sqrt{N}$, where N is the number of applications.
- If they are correlated, the sum can grow like N .

In standard 32-bit floating-point arithmetic (FP32), this is usually manageable. However, with lower precision formats like FP16 or FP8, this can lead to numerical overflow. Techniques like **mixed-precision training** (where intermediate sums are kept in a higher precision format) are used to mitigate this.

9.3 Pooling and Downsampling

Pooling layers reduce the spatial dimensions of the feature maps, which helps to reduce the number of parameters and computations, and also provides a degree of translation invariance.

- **Striding:** The simplest form of downsampling. Instead of sliding the convolutional filter one pixel at a time, we move it by a stride of $S > 1$. For example, a stride of 2 keeps only one out of every four outputs in a 2x2 region. Backpropagation is straightforward.
- **Mean Pooling:** The output is the average of the values in a pooling window (e.g., a 2×2 region).

$$\text{Output} = \frac{1}{|\text{Pooling domain}|} \sum_{i \in \text{Pooling domain}} h_i$$

This is equivalent to a convolution with a fixed (non-learnable) filter where all weights are $\frac{1}{|\text{Pooling domain}|}$. During backpropagation, the gradient from the output is distributed equally among all the inputs in the pooling window.

- **Max Pooling:** The output is the maximum value in the pooling window.

$$\text{Output} = \max_{i \in \text{Pooling domain}} h_i$$

The intuition is to capture the presence of a feature. If a "finger detector" fires strongly anywhere in the region, the max pool output will be high.

- **Backpropagation for Max Pooling:** The gradient is passed back only to the input unit that had the maximum value (the 'argmax'). All other inputs in the window receive a gradient of zero. This is because infinitesimal changes to non-maximal values do not affect the output of the max function. The operation acts as a "gradient router".

9.4 Classic CNN Architectures

Early CNNs combined these building blocks in a characteristic pattern.

- **AlexNet (2012):** A seminal network that popularized deep CNNs.
 - **Structure:** A series of convolutional layers (often with non-linearity like ReLU), followed by max pooling layers. This pattern reduces the spatial dimensions ($224 \times 224 \rightarrow 55 \times 55 \rightarrow 27 \times 27 \dots$) while increasing the number of channels ($3 \rightarrow 96 \rightarrow 256 \dots$).
 - **End:** After several conv/pool blocks, the final feature map is flattened into a single vector and fed into a few fully connected (MLP) layers, ending in a softmax output for classification.
- **VGG (2014):** Followed a similar philosophy but used a more uniform architecture with stacks of small (3×3) convolutional filters, demonstrating that depth was a key component for performance. A stack of 3×3 convolutional layers has more non-linearities than a single larger filter, increasing expressive power.

9.5 Data Augmentations

CNNs build in translation invariance through weight sharing. To achieve invariance to other transformations (like rotation, scaling), we use data augmentation. The idea is to artificially expand the

training dataset by creating modified copies of the data.

The goal is to shape the **inductive bias** of the model—the types of patterns it prefers to learn. This can be done via architecture (weight sharing), optimization, or data augmentation.

9.5.1 Basic Augmentations

These augmentations aim to make the training distribution better reflect the true data distribution one might encounter. They are typically applied randomly to each batch during training.

- **autocontrast, equalize:** Adjusting image contrast and brightness.
- **rotate, shear_x/y, translate_x/y:** Geometric transformations.
- **posterize, solarize:** Color and pixel value transformations.

With these augmentations, the network rarely sees the exact same input twice, even over many epochs.

9.5.2 Aggressive Augmentations for Robustness

These methods modify the training data in ways not typically expected in the test data. The goal is to make the training task harder, forcing the model to learn more robust and generalizable features.

- **Cutout:** Randomly removes a square patch from an image, forcing the model to use context rather than relying on a single salient feature.
- **Mixup:** Linearly interpolates between two images and their labels. For images x_1, x_2 with one-hot labels y_1, y_2 :

$$x_{new} = \lambda x_1 + (1 - \lambda)x_2 \quad , \quad y_{new} = \lambda y_1 + (1 - \lambda)y_2$$

- **CutMix:** A patch from one image is cut and pasted onto another image. The labels are mixed proportionally to the area of the patches.
- **PixMix:** A more complex method that mixes an image with various "psychedelic" fractal images to act as structured noise, forcing the model to ignore compelling but irrelevant distractors.

9.6 Training Stability: Normalization Layers

A deep network can suffer from training instability. As activations and gradients propagate forward and backward through many layers, their magnitudes can explode (leading to 'NaN's) or vanish. This is especially true for deep networks where effects multiply.

Normalization layers are inserted between other layers to control the distribution of activations. They act as "speed bumps" for activations and gradients, ensuring they stay in a reasonable range. The high-level idea is to perform something akin to input standardization, but for the inputs to intermediate layers.

9.6.1 RMS Norm Layer

This is a simple normalization layer motivated by ensuring the assumptions of schemes like Maximal Update Parametrization (MuP) hold throughout training. MuP assumes inputs have an RMS norm of 1. While initialization can ensure this, the norm can drift during training. An RMS Norm layer explicitly enforces it.

- Given a hidden activation vector $\vec{h}$, the normalized output $\tilde{\vec{h}}$ is:

$$\tilde{\vec{h}} = \frac{\vec{h}}{\sqrt{\frac{1}{d} \sum_{j=1}^d h_j^2 + \epsilon}} = \frac{\vec{h}}{\text{RMS}(\vec{h})\sqrt{d} + \epsilon}$$

where d is the dimension of the vector.

- The output $\tilde{\vec{h}}$ will have an RMS norm of 1 (ignoring ϵ).
- This version has no learnable parameters and is effective with certain modern optimizers.

9.6.2 Batch Normalization (BatchNorm)

One of the first and most widely used normalization layers. It standardizes the activations for each channel by computing the mean and standard deviation over a mini-batch of data.

For a feature map in a CNN, we have dimensions for batch (N), channels (C), height (H), and width (W). BatchNorm computes statistics by averaging over the batch and spatial dimensions (N, H, W) independently for each channel.

Let $\mathcal{B}$ be the set of activations for a single channel over a mini-batch (i.e., across N, H, and W).

1. **Compute mini-batch mean:**

$$\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x$$

2. **Compute mini-batch variance:**

$$\sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \mu_{\mathcal{B}})^2$$

3. **Normalize:**

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

This normalizes the activations to have zero mean and unit variance over the mini-batch.

4. **Scale and Shift:** The network might need to learn a different distribution. To allow this, BatchNorm introduces two learnable parameters per channel, γ (scale) and δ (shift).

$$y = \gamma \hat{x} + \delta$$

If the network learns $\gamma = \sqrt{\sigma_{\mathcal{B}}^2}$ and $\delta = \mu_{\mathcal{B}}$, it can recover the original activation, so no expressive power is lost.